UCAB PREMIUM

A Full-Stack MERN-Based Cab Booking Web Application

A Project Report

Submitted in partial fulfilment of the requirements for the internship component of

Bachelor of Technology

in

Electronics and Communication Engineering

Submitted by

PAIDIPATI MANIKUMAR

Roll Number: 24AK5A0417

Semester: IV-I | Academic Year: 2024–2027

Internship: Full Stack Development (MERN Stack) – SmartBridge Platform

Project Duration: May 2026 – July 2026

Department of Electronics and Communication Engineering

ANNAMACHARYA INSTITUTE OF TECHNOLOGY & SCIENCES

(Affiliated to JNTUA, Anantapuramu)

2024 – 2027

ABSTRACT

UCab Premium is a full-stack cab booking web application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) as the capstone project of a Full Stack Development internship undertaken on the SmartBridge platform. The system is designed to digitally streamline the process of cab booking by connecting three distinct categories of users — Riders, Drivers, and Administrators — within a single, unified platform.

The application implements secure authentication and authorization using JSON Web Tokens (JWT), along with role-based access control to ensure that each user type is restricted to the functionalities relevant to their role. At the core of the system lies a five-stage booking lifecycle state machine that manages the progression of a ride request from initiation to completion, ensuring consistency and reliability across all booking transactions. The system also incorporates real-time driver availability management, wherein drivers transition automatically between Available, Busy, and Offline states based on their current ride status, and administrators are restricted to assigning rides only to drivers who are currently available.

The backend is architected using the Model-View-Controller (MVC) pattern and exposes thirty-five RESTful API endpoints spread across five resource groups, backed by five MongoDB collections modelled through Mongoose schemas. The frontend, comprising twenty-three React pages organised across user, driver, and administrator modules, offers dedicated dashboards for each role. Administrative dashboards feature ten key statistical cards and interactive analytics charts built using Recharts, while riders are provided with automatically generated PDF ride receipts using PDFKit. The user interface follows a premium dark navy and amber design theme, enhanced with smooth animations, skeleton loaders, and modal-based interactions to deliver a polished, professional user experience.

This project demonstrates the practical application of modern full-stack web development principles, including RESTful API design, state management, secure authentication, role-based system design, and responsive user interface development, and reflects the skills acquired during the internship period from May 2026 to July 2026.

Keywords: *MERN Stack, Cab Booking System, MongoDB, Express.js, React.js, Node.js, JWT Authentication, Role-Based Access Control, RESTful API, Booking Lifecycle State Machine.*

TABLE OF CONTENTS

1. Introduction

1.1 Background

Urban transportation has undergone a significant transformation over the past decade, largely driven by the proliferation of smartphone technology and internet connectivity. Traditional modes of hiring a cab — such as flagging one down on the street, telephoning a local taxi stand, or relying on unorganised auto-rickshaw services — are increasingly being replaced by structured, technology-driven, on-demand mobility platforms. These platforms allow a passenger to request a ride from virtually any location, track the assigned vehicle in real time, and pay digitally, all through a single mobile or web application.

The success of global platforms such as Uber, Ola, and Lyft has demonstrated that a well-designed cab booking system can simultaneously solve two problems: it gives passengers (riders) a convenient, transparent, and reliable way to book transportation, and it gives drivers a structured channel through which to find paying customers without needing to search for them manually. Replicating even a simplified version of such a system requires the integration of several complex software engineering concepts — real-time data handling, secure authentication, role-based workflows, relational-style data modelling within a NoSQL database, and a responsive, intuitive user interface.

"UCab Premium" was conceived and developed as the capstone project of a Full Stack Development Internship undertaken on the SmartBridge platform, using the MERN Stack — MongoDB, Express.js, React.js, and Node.js. The project was completed over the internship duration of May 2026 to July 2026 and was designed not merely as an academic exercise, but as a functionally complete, production-style web application capable of handling the end-to-end cab booking process for three distinct categories of users: Riders, Drivers, and Administrators.

1.2 Motivation

The motivation behind selecting a cab booking system as the internship capstone project stemmed from its suitability as a comprehensive learning vehicle for full-stack web development. Unlike simpler CRUD-based applications, a cab booking platform inherently demands:

- Multiple, interdependent user roles, each with distinct permissions and workflows.

- A well-defined state machine to govern the lifecycle of a booking from request to completion.
- Real-time-like updates to reflect the changing availability status of drivers.
- Secure, token-based authentication and strict role-based access control (RBAC).
- A properly normalised, relationally-aware data model built on a NoSQL database.
- A polished, professional user interface capable of presenting analytical data meaningfully.

Building such a system from the ground up provided practical exposure to the entire software development lifecycle — from requirement analysis and database design through to deployment-ready, production-styled implementation — making it an ideal project for consolidating and demonstrating full-stack development competence.

1.3 Overview of the MERN Stack

The MERN Stack is a JavaScript-based technology stack used for building full-stack web applications entirely using a single programming language across both the client and server sides. It is composed of four principal technologies, summarised in Table 1.1.

Layer	Technology	Role in UCab Premium
Database	MongoDB	NoSQL document database used to store users, drivers, bookings, and system data.
Backend Framework	Express.js	Provides the routing and middleware layer for the 35 RESTful API endpoints.
Frontend Library	React.js	Powers the 23-page single-page application across rider, driver, and admin modules.
Runtime Environment	Node.js	Executes JavaScript on the server, hosting the Express application.

Table 1.1: Components of the MERN Stack and their role in UCab Premium

The choice of the MERN Stack allows a single development team — or, in this case, a single developer — to work across the entire application using one language (JavaScript/JSX), significantly reducing context-switching overhead and enabling faster iteration between the frontend and backend during development.

1.4 Problem Context

In the absence of a structured digital platform, cab booking within smaller organisations, campuses, or local service providers is often handled manually — through phone calls, messaging applications, or in-person coordination. This approach suffers from a lack of transparency, no formal record-keeping, inefficient driver allocation, and an inability to monitor operational metrics such as ride volume, revenue, or driver performance. UCab Premium addresses this gap by providing a self-contained, digitally managed booking ecosystem that automates driver assignment, tracks the complete lifecycle of every ride, and gives administrators real-time visibility into system-wide operations through analytical dashboards.

2. Literature Survey

A literature survey was conducted to understand the existing body of work in the domain of ride-hailing and cab booking systems, covering both commercially deployed platforms and academically documented systems. This survey informed several key design decisions in UCab Premium, particularly around booking state management, role separation, and driver allocation logic.

2.1 Review of Existing Platforms and Research

Reference / Platform	Focus Area	Key Observations
Uber Technologies – Ride-Hailing Platform	Real-time driver matching and dynamic pricing	Demonstrated the value of automated, proximity-based driver assignment and the importance of a clearly defined ride-status lifecycle for both rider and driver applications.
Ola Cabs – Ride-Hailing Platform	Multi-role application architecture	Highlighted the need for entirely separate application experiences for riders and drivers, while sharing a common backend and booking data model.
Academic studies on NoSQL schema design for transactional apps	Document-oriented data modelling	Emphasised the trade-offs between embedding and referencing related data in MongoDB, which directly informed the design of the Booking, User, and Driver collections in UCab Premium.
Research on state machines in workflow-driven systems	Lifecycle and status management	Supported the adoption of an explicit, finite-state booking lifecycle (Requested → Accepted → Ongoing → Completed / Cancelled) rather than an ad hoc status field.
Studies on JWT-based authentication in SPAs	Security and session management	Confirmed JSON Web Tokens as a suitable stateless authentication mechanism for React single-page applications communicating with an Express REST API.

Table 2.1: Summary of literature and platforms reviewed

2.2 Key Insights Derived from the Survey

- Role-based separation is essential: successful ride-hailing systems maintain clearly isolated experiences for riders, drivers, and administrators, even though they share a common backend.

- A well-defined booking lifecycle, represented as a finite state machine, reduces data inconsistency and simplifies both backend validation and frontend status rendering.
- Driver availability must be treated as a first-class, tightly controlled property, since incorrect assignment of a busy driver directly degrades service reliability.
- Token-based authentication (JWT) is the prevailing industry standard for securing REST APIs consumed by single-page applications, offering statelessness and scalability.
- Administrative dashboards with visual analytics (charts, statistic cards) significantly improve an operator's ability to monitor system health at a glance.

These insights collectively shaped the architectural and functional decisions detailed in the subsequent chapters of this report, particularly the design of the booking state machine, the driver availability model, and the administrator analytics dashboard.

3. Existing System

3.1 Description of the Existing System

In the absence of a dedicated software platform, cab booking within most small-to-medium scale operations — including local taxi services, campus transport arrangements, and informal driver networks — continues to rely on manual or semi-manual processes. A rider typically contacts a driver or a dispatch coordinator directly through a phone call or a generic messaging application. The coordinator then manually attempts to identify a driver who is free, communicates the trip details verbally or via text, and has no systematic means of tracking whether the ride was actually completed, cancelled, or disputed.

Where basic digital tools are used, they are usually limited to spreadsheets or shared documents used to log bookings after the fact, rather than to drive the booking process itself. This results in a system that is reactive rather than proactive, and one that offers no real-time insight into fleet utilisation or operational performance.

3.2 Limitations of the Existing System

Limitation	Impact
No centralised booking record	Ride history is scattered across phone logs and personal notes, making auditing and dispute resolution difficult.
Manual driver allocation	Coordinators may assign a ride to a driver who is already occupied, leading to delays or cancellations.
No defined booking lifecycle	Ride status (requested, ongoing, completed) is not formally tracked, causing ambiguity about a trip's current state.
Absence of role-based access	Any party with access to shared logs can view or alter data, with no enforced separation between rider, driver, and administrator privileges.
No analytics or reporting	Operators cannot easily determine ride volume, revenue trends, or driver performance without manual computation.
No digital receipts	Riders receive no formal proof of payment or trip completion, reducing transparency and trust.
Poor scalability	The manual process breaks down rapidly as the number of riders and drivers increases beyond what a single coordinator can track.

Table 3.1: Limitations of the existing manual/semi-manual system

4. Proposed System

4.1 Overview

UCab Premium is proposed as a fully digital, self-contained cab booking platform that replaces manual coordination with an automated, rule-driven system. It formalises the entire ride process — from booking request to payment receipt — into a structured workflow governed by a defined booking lifecycle state machine, enforced through a secure, role-based backend API and presented through a modern, responsive React-based interface.

4.2 Key Features of the Proposed System

- JWT-based authentication with strict role-based access control for Riders, Drivers, and Administrators.
- A five-stage booking lifecycle state machine ensuring every ride progresses through well-defined, validated states.
- Automatic driver availability management, cycling drivers between Available, Busy, and Offline states as bookings progress.
- Administrator-controlled driver assignment, restricted to drivers who are currently marked as available.
- A comprehensive administrator dashboard with ten statistical summary cards and Recharts-based analytical visualisations.
- Automated PDF ride-receipt generation using PDFKit, giving riders a formal, downloadable proof of each completed trip.
- A cohesive, premium dark navy and amber user interface with animations, skeleton loaders, and modal-driven interactions.
- Thirty-five RESTful API endpoints, organised into five resource groups, backed by an MVC-structured Express backend.

4.3 Existing System vs. Proposed System

Parameter	Existing (Manual) System	Proposed System – UCab Premium
Booking Method	Phone call / message, manually logged	Structured in-app booking with a real-time-tracked lifecycle
Driver Allocation	Manual, prone to double-booking	Automated, restricted to available drivers only

Parameter	Existing (Manual) System	Proposed System – UCab Premium
Access Control	No formal separation of roles	JWT-based authentication with strict RBAC (Rider/Driver/Admin)
Ride Status Tracking	Not formally tracked	Five-stage state machine (Requested to Completed/Cancelled)
Reporting & Analytics	Manual, after-the-fact calculation	Real-time dashboard with 10 stat cards and Recharts visual analytics
Proof of Trip / Payment	None, or informal notes	Auto-generated PDF receipt via PDFKit
Scalability	Breaks down with growing volume	Designed to scale via a normalised MongoDB schema and REST API
User Experience	Inconsistent, dependent on coordinator	Consistent, dedicated interfaces for each of the 23 React pages

Table 4.1: Comparative analysis of the existing system and the proposed UCab Premium system

4.4 Advantages of the Proposed System

- Eliminates double-booking of drivers through enforced availability checks at the point of assignment.
- Provides complete transparency and traceability for every booking through its recorded lifecycle.
- Reduces administrative overhead by automating driver-status transitions rather than requiring manual updates.
- Offers data-driven decision-making support through dashboard analytics for administrators.
- Improves rider trust and satisfaction through digital receipts and a polished, professional interface.

5. Objectives

The objectives of the UCab Premium project are categorised into primary objectives, which define the core functional goals of the system, and secondary objectives, which address supporting, quality-oriented goals.

5.1 Primary Objectives

- To design and implement a full-stack, role-based cab booking system using the MERN Stack.
- To develop a secure authentication and authorization mechanism using JSON Web Tokens (JWT) for Riders, Drivers, and Administrators.
- To implement a five-stage booking lifecycle state machine that governs the complete lifecycle of a ride request.
- To build an automated driver availability management system that transitions drivers between Available, Busy, and Offline states.
- To provide administrators with the ability to assign rides exclusively to available drivers, preventing scheduling conflicts.
- To develop a functionally complete REST API comprising 35 endpoints across five resource groups, following MVC architecture.

5.2 Secondary Objectives

- To design an administrator dashboard featuring 10 statistical cards and Recharts-based analytical visualisations.
- To implement automated PDF receipt generation for completed rides using PDFKit.
- To create a premium, cohesive user interface using a dark navy and amber design theme, incorporating animations, skeleton loaders, and modals.
- To ensure the application's data model, built on five MongoDB collections, supports efficient querying and maintains referential consistency.
- To document the complete system through formal, university-standard project documentation.

6. Scope of the Project

6.1 Functional Scope

The scope of UCab Premium encompasses the complete cycle of cab booking operations for three categories of users, implemented as a working web application rather than a conceptual prototype. The functional scope includes rider registration and ride booking, driver registration and ride management, and administrator oversight of the entire platform, including driver assignment and system-wide analytics.

Module	In Scope
Rider Module	Registration/login, ride booking, ride history, PDF receipt download, ride status tracking.
Driver Module	Registration/login, availability toggling, accepting/updating assigned rides, ride dashboard.
Admin Module	Driver assignment, user/driver management, analytics dashboard, search and filter tools.
Backend / API	35 RESTful endpoints across five resource groups, JWT authentication, MVC architecture.
Database	Five MongoDB collections modelled with Mongoose schemas covering users, drivers, and bookings.

Table 6.1: Functional scope of UCab Premium by module

6.2 Out of Scope

Certain features, while common in commercial ride-hailing platforms, were considered beyond the scope of this internship project due to time constraints and the academic nature of the internship. These include:

- Live GPS-based map tracking of drivers and real-time route navigation.
- Integration with third-party payment gateways for actual monetary transactions.
- Dynamic, demand-based fare calculation (surge pricing).
- Native mobile applications for iOS and Android (the system is delivered as a responsive web application).
- Multi-language and multi-currency support.

6.3 Future Scope

While outside the current implementation, the architecture of UCab Premium has been designed to reasonably accommodate future extensions, including real-time GPS tracking through WebSocket integration, payment gateway integration, a dedicated mobile application built with React Native, and machine-learning-based demand prediction for smarter driver allocation. These potential enhancements are discussed further in the Future Enhancements chapter of this report.

7. Software Requirements

The development and execution of UCab Premium requires a specific set of software tools, frameworks, and runtime environments. Table 7.1 summarises the software requirements for both the development and deployment environments.

Category	Requirement	Purpose
Operating System	Windows 10/11, macOS, or Linux	Development machine OS; the application itself is OS-independent.
Runtime Environment	Node.js (v18 LTS or higher)	Executes the Express backend server and supports the React build tooling.
Package Manager	npm	Installs and manages backend and frontend dependencies.
Database	MongoDB Atlas (cloud-hosted)	Stores all application data as document collections.
Code Editor	Visual Studio Code	Primary IDE used for writing and debugging the application.
Version Control	Git and GitHub	Source code versioning and repository hosting.
API Testing Tool	Postman	Used to test and validate the 35 REST API endpoints during development.
Browser	Google Chrome / Microsoft Edge (latest)	Used for running and testing the React frontend, including DevTools.
Backend Framework	Express.js (v4)	Provides routing, middleware, and REST API structure.
Frontend Library	React.js (v18)	Builds the component-based single-page application.

Table 7.1: Software requirements for development and deployment

8. Hardware Requirements

As UCab Premium is a web-based application, its hardware requirements are modest on both the development and client sides. Table 8.1 outlines the minimum and recommended hardware specifications.

Component	Minimum Requirement	Recommended
Processor	Dual-core, 1.6 GHz	Quad-core, 2.4 GHz or higher
RAM	4 GB	8 GB or higher
Storage	5 GB free disk space	10 GB or higher (SSD preferred)
Internet Connectivity	Stable broadband (for MongoDB Atlas access)	Broadband, 10 Mbps or higher
Display	1366 × 768 resolution	1920 × 1080 (Full HD) or higher
Client Device	Any device with a modern web browser	Laptop/desktop or smartphone with an updated browser

Table 8.1: Hardware requirements for development and end-user access

Since MongoDB Atlas is a fully managed, cloud-hosted database service, no dedicated database server hardware is required locally. This significantly reduces the infrastructure burden on both the developer's machine and the end user's device, as all data persistence occurs on Atlas's managed cloud infrastructure.

9. Functional Requirements

Functional requirements describe the specific behaviours and features that UCab Premium must provide to its three user roles. Table 9.1 lists the core functional requirements, grouped by module.

Module	Functional Requirement
Authentication	The system shall allow users to register and log in securely using JWT-based authentication, with role-based redirection (Rider/Driver/Admin).
Rider	The system shall allow riders to create a new ride booking by specifying pickup and drop locations.
Rider	The system shall allow riders to view their complete ride history and download a PDF receipt for completed rides.
Driver	The system shall allow drivers to toggle their availability status between Available and Offline.
Driver	The system shall allow drivers to accept, start, and complete rides assigned to them.
Admin	The system shall allow administrators to assign a pending booking to any driver currently marked as Available.
Admin	The system shall allow administrators to search and filter users, drivers, and bookings.
Admin	The system shall display a dashboard with 10 statistical summary cards and analytical charts.
Booking Lifecycle	The system shall enforce a five-stage booking lifecycle (Requested → Accepted → Ongoing → Completed / Cancelled).
Notifications	The system shall reflect real-time status changes of bookings and driver availability within the interface.

Table 9.1: Functional requirements of UCab Premium, grouped by module

10. Non-Functional Requirements

Non-functional requirements define the quality attributes that govern how the system performs its functions, rather than what functions it performs. These requirements are equally critical to the overall usability and reliability of UCab Premium.

Requirement	Description
Performance	API responses for standard CRUD operations should complete within 1–2 seconds under normal load.
Security	All passwords must be hashed using bcryptjs; all protected routes must validate JWT tokens before granting access.
Scalability	The MongoDB schema and REST API design should support a growing number of users, drivers, and bookings without structural redesign.
Usability	The interface should be intuitive, visually consistent, and require minimal learning for each user role.
Availability	The application, hosted on cloud infrastructure, should aim for high uptime with minimal service interruption.
Maintainability	The codebase should follow the MVC architecture and modular component design to simplify future updates.
Portability	The web application should function consistently across major modern browsers and screen sizes.
Data Integrity	Mongoose schema validation should prevent invalid or incomplete data from being persisted to the database.

Table 10.1: Non-functional requirements of UCab Premium

11. Feasibility Study

A feasibility study was conducted prior to development to assess whether UCab Premium could be realistically built and deployed within the given internship timeframe (May 2026 – July 2026), using the proposed technology stack. The study examined the project across three standard dimensions: technical, economic, and operational feasibility.

11.1 Technical Feasibility

The MERN Stack is a mature, well-documented, and widely adopted technology stack with extensive community support, making it technically feasible for a single developer to design, implement, and test a multi-role application of this scope within the internship duration. All required tools — Node.js, MongoDB Atlas, React, and associated libraries — are freely available and require no specialised or proprietary infrastructure.

11.2 Economic Feasibility

UCab Premium was developed entirely using free-tier and open-source resources. MongoDB Atlas offers a free-tier cluster suitable for development and academic use, Node.js and all associated npm packages are open source, and development tools such as Visual Studio Code, Postman, and Git are free of cost. Consequently, the project incurred no direct monetary expenditure, making it highly economically feasible for an academic internship context.

11.3 Operational Feasibility

The system was designed with straightforward, role-specific workflows for Riders, Drivers, and Administrators, minimising the operational learning curve. Since the application is delivered as a responsive web platform accessible through any modern browser, no specialised end-user training or additional software installation is required for operation.

Feasibility Type	Assessment	Conclusion
Technical	Mature stack, strong documentation, achievable within internship timeframe	Feasible
Economic	Built entirely on free-tier and open-source tools	Feasible
Operational	Simple, role-specific workflows; browser-based access	Feasible

Table 11.1: Summary of the feasibility study

12. Technology Stack

The selection of each technology used in UCab Premium was driven by its suitability for building a secure, scalable, and maintainable full-stack web application within a JavaScript-centric ecosystem. This section explains the rationale behind each major technology choice, supported by comparative evaluation against commonly considered alternatives.

12.1 React (Frontend Library)

React was selected for building the 23-page frontend of UCab Premium due to its component-based architecture, which allows the rider, driver, and admin interfaces to be built as independent, reusable units. Its virtual DOM enables efficient re-rendering, which is particularly beneficial for dynamically updating dashboards, ride statuses, and analytics charts without full-page reloads. React's vast ecosystem also provides seamless integration with charting libraries such as Recharts, used extensively in the admin analytics dashboard.

Framework	Learning Curve	Performance	Ecosystem	Suitability for UCab Premium
React	Moderate	High (Virtual DOM)	Extensive	Selected — ideal for component-driven, role-based dashboards
Angular	Steep	High	Large but opinionated	More suited to large enterprise apps; heavier for this scope
Vue.js	Easy	High	Growing but smaller	Viable alternative, but smaller community support

Table 12.1: Comparison of frontend frameworks considered

12.2 Node.js and Express.js (Backend)

Node.js was chosen as the server-side runtime because it allows JavaScript to be used consistently across both the frontend and backend, reducing context-switching and enabling code and logic sharing where appropriate. Its non-blocking, event-driven architecture is well suited to handling multiple concurrent API requests, such as simultaneous ride bookings and driver status updates. Express.js was layered on top of Node.js to provide a minimal, flexible routing and middleware framework, which was used to structure all 35 REST API endpoints following the MVC architectural pattern.

Backend Option	Language	Concurrency Model	Suitability for UCab Premium
Node.js + Express.js	JavaScript	Non-blocking, event-driven	Selected — unifies language across stack, lightweight and fast to develop
Django (Python)	Python	WSGI-based, synchronous by default	Strong ORM but introduces a second language into the stack
Spring Boot (Java)	Java	Multi-threaded	Powerful but heavier setup, not ideal for rapid internship-scale development

Table 12.2: Comparison of backend framework options

12.3 MongoDB Atlas (Database)

MongoDB Atlas, a fully managed cloud-hosted NoSQL database service, was selected for its flexible, document-oriented data model, which aligns naturally with JavaScript object structures used throughout the MERN Stack. Its schema flexibility was particularly useful for modelling the varying attributes of Users, Drivers, and Bookings across the five MongoDB collections, while Mongoose was used on top of it to enforce structured schema validation. Atlas also eliminates the need for local database server management, providing built-in scalability, automated backups, and remote accessibility — all valuable for a cloud-deployable application.

12.4 JWT – JSON Web Tokens (Authentication)

JWT was adopted as the authentication mechanism because it is inherently stateless — the server does not need to maintain session data, as each token carries the necessary user identity and role information, signed and verified on every request. This makes JWT particularly well suited to a REST API architecture, where each of the 35 endpoints can independently validate a request without querying a session store, improving both scalability and simplicity of the authentication middleware.

12.5 bcryptjs (Password Security)

bcryptjs was used to hash user passwords before storing them in MongoDB. Unlike reversible encryption, bcryptjs applies a one-way, salted hashing algorithm, meaning that even in the unlikely event of a database compromise, raw passwords are never exposed.

Its built-in salting mechanism also protects against precomputed rainbow-table attacks, making it a well-established standard for password security in Node.js applications.

12.6 Axios (HTTP Client)

Axios was used on the frontend to handle all HTTP communication between the React application and the Express REST API. Compared to the native Fetch API, Axios offers a more concise syntax, automatic JSON transformation, built-in request/response interceptors — used in UCab Premium to automatically attach the JWT token to outgoing requests — and more consistent error handling across browsers.

Feature	Axios	Native Fetch API
JSON Parsing	Automatic	Manual (.json() call required)
Interceptors	Built-in support	Not supported natively
Error Handling	Rejects promise on HTTP error status	Only rejects on network failure
Browser Support	Consistent across environments	Varies slightly across older browsers

Table 12.3: Axios versus the native Fetch API

12.7 React Router (Client-Side Routing)

React Router was used to manage navigation across the 23 pages of the application without triggering full page reloads, preserving the single-page application experience. It also enabled the implementation of protected, role-based routes — ensuring that, for example, a Rider cannot navigate directly to an Admin dashboard URL, reinforcing the role-based access control enforced at the API level.

12.8 Summary of Technology Choices

Technology	Role	Primary Reason for Selection
React	Frontend library	Component-based UI for 23 role-specific pages
Node.js	Backend runtime	Unified JavaScript across the stack; non-blocking I/O
Express.js	Backend framework	Lightweight routing/middleware for 35 REST endpoints
MongoDB Atlas	Database	Flexible, cloud-hosted document storage with schema validation via Mongoose
JWT	Authentication	Stateless, scalable token-based identity

Technology	Role	Primary Reason for Selection
		verification
bcryptjs	Password security	One-way salted hashing to protect stored credentials
Axios	HTTP client	Simplified requests with interceptor-based token handling
React Router	Client-side routing	Seamless, role-protected navigation within the SPA

Table 12.4: Summary of core technologies used in UCab Premium and their selection rationale

13. System Architecture

UCab Premium follows a three-layer architecture consisting of a Client Layer, an Application Layer, and a Data Layer. This separation ensures that presentation logic, business logic, and data persistence remain independent of one another, allowing each layer to evolve or scale without directly impacting the others.

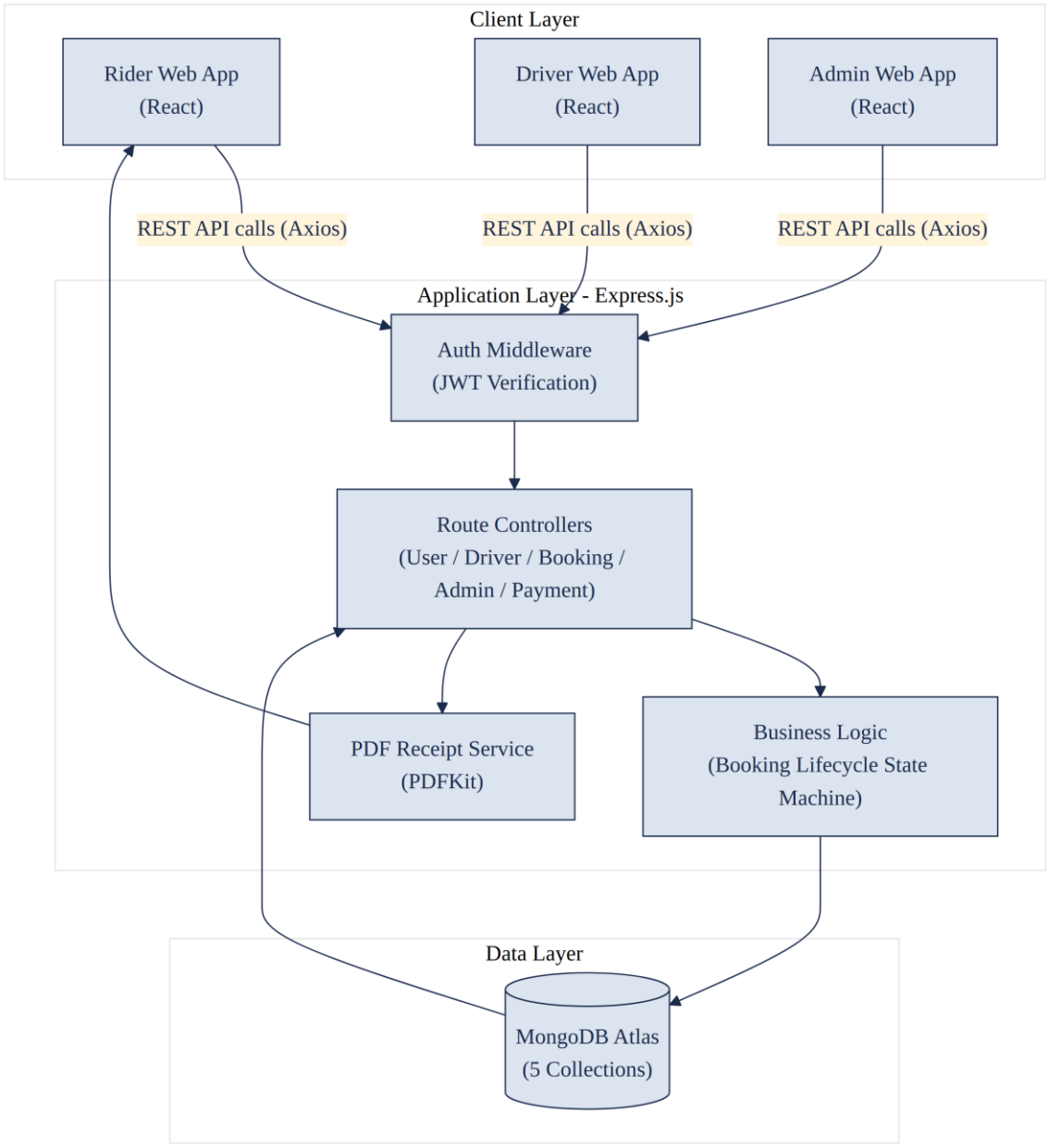

Figure 13.1: High-level system architecture of UCab Premium

13.1 Explanation of the Architecture

The Client Layer consists of three React-based web applications, functionally separated for Riders, Drivers, and Administrators, though built from a shared 23-page React

codebase organised by role. Each client communicates with the backend exclusively through REST API calls made using Axios.

The Application Layer, built with Express.js, is the core of the system. Every incoming request first passes through the Authentication Middleware, which verifies the JWT token attached to the request and confirms that the user's role is authorised to access the requested resource. Once validated, the request is handed to the appropriate Route Controller, which applies the relevant business rules — most notably the Booking Lifecycle State Machine, which governs valid transitions of a booking's status. For completed rides, the Application Layer also invokes the PDF Receipt Service, built using PDFKit, to generate a downloadable proof of trip completion.

The Data Layer consists of a single MongoDB Atlas cluster hosting five collections — Users, Drivers, Vehicles, Bookings, and Payments — all accessed through Mongoose object models defined in the Application Layer. This layered design ensures that the frontend never interacts with the database directly, preserving a clean separation of concerns and a single, controlled point of data validation and access.

14. MVC Architecture

The backend of UCab Premium is structured according to the Model-View-Controller (MVC) architectural pattern, which separates data handling, business logic, and response formatting into distinct, well-defined components. This structure was chosen to keep the 35-endpoint REST API organised, testable, and maintainable as the application grew across its two development phases.

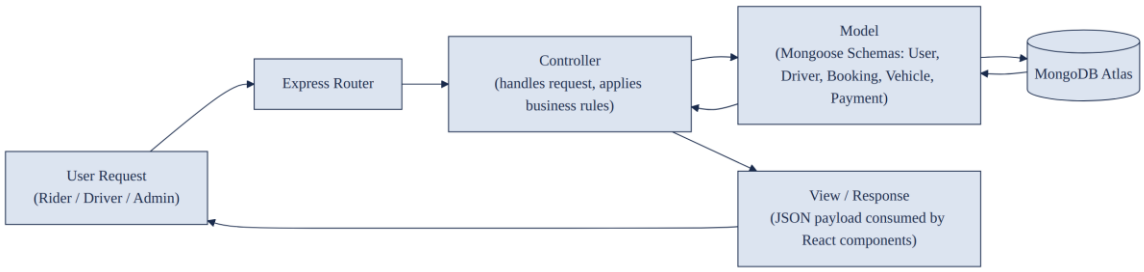

Figure 14.1: MVC request-response flow within the Express backend

14.1 Explanation of the MVC Flow

Component	Responsibility in UCab Premium
Model	Mongoose schemas (User, Driver, Booking, Vehicle, Payment) defining structure, validation rules, and interaction with MongoDB Atlas.
Controller	Route-handler functions that process incoming requests, apply business logic such as the booking state machine, and invoke the appropriate Model methods.
View	Since UCab Premium exposes a REST API rather than server-rendered pages, the “View” is the JSON response returned to the React frontend, which independently renders the UI.

Table 14.1: Mapping of MVC components to UCab Premium's backend

When a request is received, the Express Router directs it to the corresponding Controller based on the endpoint and HTTP method. The Controller then interacts with the relevant Model to read or write data in MongoDB Atlas. Once the Model operation completes, the Controller formats the result into a JSON response, which is sent back to the React client and treated as the View layer of the pattern. This flow keeps route-handling, data-validation, and business-rule logic cleanly separated across dedicated files within the backend's directory structure.

15. User Flow

The user flow diagram below traces the journey of a Rider from initial login through to the completion of a ride and the generation of a PDF receipt, illustrating how the booking lifecycle state machine governs the ride's progress at each stage.

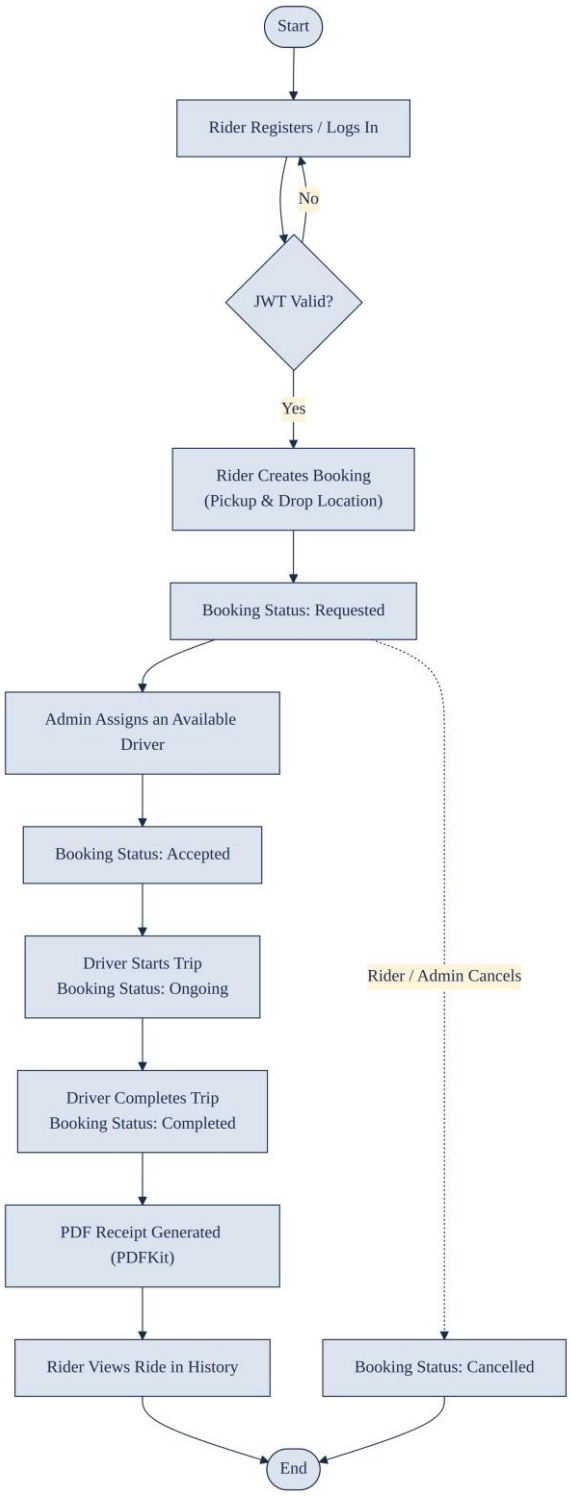

Figure 15.1: End-to-end rider booking user flow

15.1 Explanation of the User Flow

The flow begins when a Rider registers or logs into the platform; if the submitted credentials fail JWT-based authentication, the user is returned to the login step. Once authenticated, the Rider creates a booking by specifying pickup and drop locations, placing the booking into the Requested state. An Administrator then reviews pending bookings and assigns an available Driver, moving the booking into the Accepted state and simultaneously updating that driver's availability to Busy.

Once the assigned Driver begins the trip, the booking transitions to the Ongoing state; upon trip completion, it moves to the Completed state, at which point the system automatically generates a PDF receipt using PDFKit and updates the driver's availability back to Available. The completed ride, along with its receipt, then becomes visible in the Rider's ride history. At any point prior to completion, either the Rider or an Administrator may cancel the booking, moving it directly to the Cancelled state and ending the flow.

16. Use Case Diagram

The use case diagram identifies the three actors of UCab Premium — Rider, Driver, and Administrator — and the distinct set of system functionalities (use cases) each actor is authorised to perform, as enforced by the role-based access control implemented at the API level.

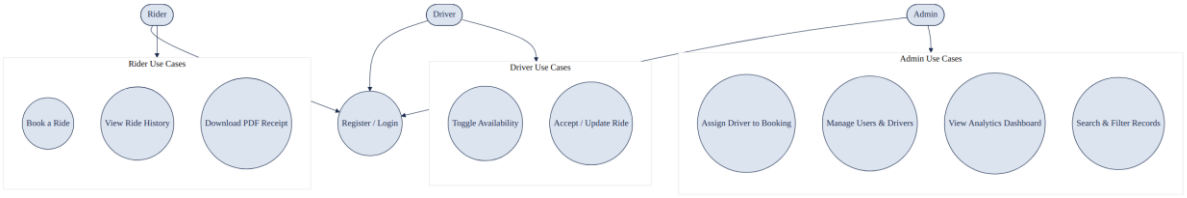

Figure 16.1: Use case diagram showing actor-specific system functionality

16.1 Explanation of the Use Case Diagram

All three actors share a common entry point — Register / Login — which authenticates the user and issues a role-specific JWT token. From this shared point, each actor is restricted to a distinct group of use cases. The Rider group covers booking a ride, viewing ride history, and downloading a PDF receipt. The Driver group covers toggling availability and accepting or updating an assigned ride. The Administrator group covers assigning drivers to bookings, managing users and drivers, viewing the analytics dashboard, and searching or filtering platform records.

This clear separation of use cases by actor reflects the role-based access control (RBAC) enforced throughout the backend: even if a request reaches the API, the authentication middleware rejects any attempt by a user to invoke a use case outside their assigned role.

17. Sequence Diagram

The sequence diagram below illustrates the interaction between the Rider, the React frontend, the Express API, MongoDB Atlas, the Driver, and the Administrator across the complete lifecycle of a single booking — from creation through to completion and receipt generation.

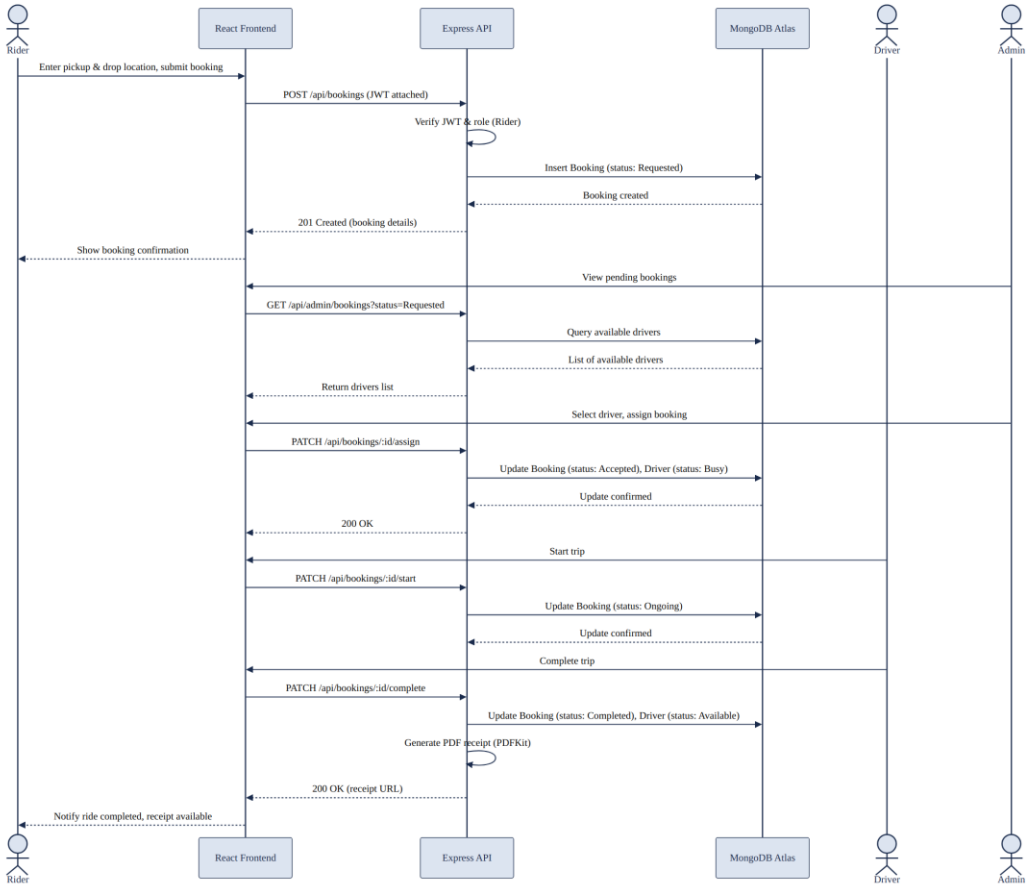

Figure 17.1: Sequence diagram for the ride booking and completion process

17.1 Explanation of the Sequence Diagram

The sequence begins when the Rider submits a booking request through the React frontend, which sends a POST request to the /api/bookings endpoint with the JWT token attached. The API verifies the token and the Rider role before inserting a new Booking document into MongoDB Atlas with a status of Requested, and returns the created booking details to the frontend.

The Administrator subsequently queries the list of pending bookings and available drivers, selects a suitable driver, and triggers an assignment request. The API updates the Booking status to Accepted and simultaneously updates the assigned Driver's status to

Busy. When the Driver starts the trip through their dashboard, the API updates the Booking status to Ongoing; upon completion, it updates the status to Completed, resets the Driver's availability to Available, and invokes PDFKit to generate a PDF receipt. The API's final response includes a reference to this receipt, which the frontend uses to notify the Rider that the ride is complete and the receipt is ready for download.

18. Deployment Architecture

UCab Premium is designed for a cloud-based deployment model in which the frontend, backend, and database are hosted independently, communicating over HTTPS. This separation allows each layer to be scaled, updated, or redeployed without affecting the others.

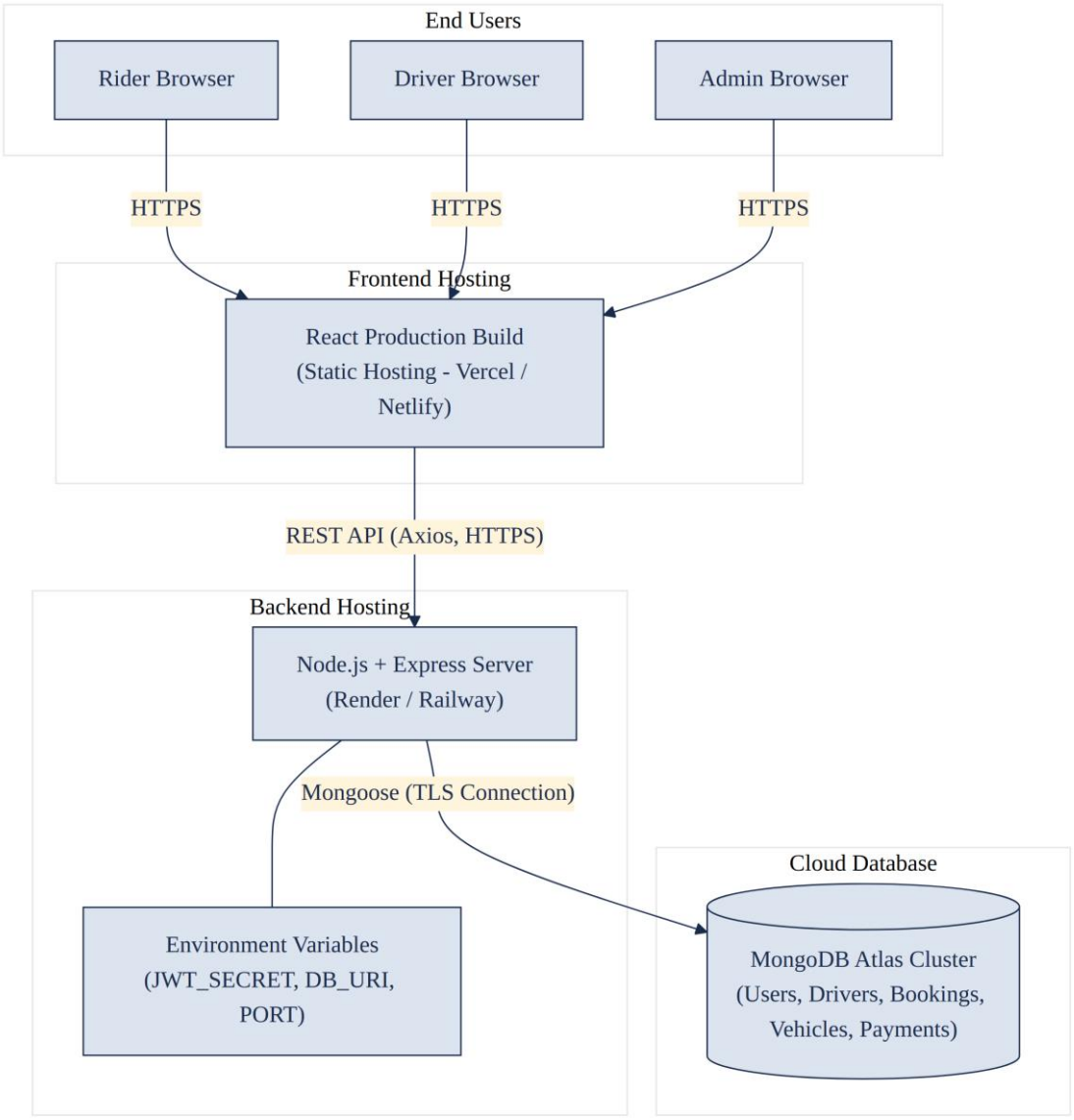

Figure 18.1: Deployment architecture of UCab Premium across hosting environments

18.1 Explanation of the Deployment Architecture

End users — Riders, Drivers, and Administrators — access the application through a standard web browser over HTTPS. The React frontend is compiled into a static production build and served through a static hosting provider such as Vercel or Netlify, offering fast global content delivery without requiring a dedicated server.

The Express backend is deployed as a Node.js service on a platform such as Render or Railway, configured with environment variables including the JWT signing secret, the MongoDB Atlas connection string, and the application port. All communication between the frontend and backend occurs through Axios-driven REST API calls over HTTPS. The backend connects to the MongoDB Atlas cluster — which independently hosts the Users, Drivers, Vehicles, Bookings, and Payments collections — using a secure, TLS-encrypted Mongoose connection, ensuring that data in transit between the application server and the database remains protected.

19. Folder Structure

The UCab Premium codebase is organised into two top-level directories — client and server — reflecting the separation between the React frontend and the Express backend. Each directory follows a modular structure aligned with the MVC pattern and the three user roles.

19.1 Backend (server) Folder Structure

```
server/
|-- config/
|   `-- db.js                  // MongoDB Atlas connection setup
|-- models/
|   |-- User.js
|   |-- Driver.js
|   |-- Vehicle.js
|   |-- Booking.js
|   `-- Payment.js
|-- controllers/
|   |-- authController.js
|   |-- riderController.js
|   |-- driverController.js
|   |-- adminController.js
|   `-- bookingController.js
|-- routes/
|   |-- authRoutes.js
|   |-- riderRoutes.js
|   |-- driverRoutes.js
|   |-- adminRoutes.js
|   `-- bookingRoutes.js
|-- middleware/
|   |-- authMiddleware.js       // JWT verification
|   `-- roleMiddleware.js       // Role-based access control
|-- utils/
|   `-- generateReceipt.js      // PDFKit receipt generation
|-- .env
|-- server.js
`-- package.json
```

19.2 Frontend (client) Folder Structure

```
client/
|-- public/
|-- src/
|   |-- assets/
|   |-- components/
|   |   |-- common/            // Navbar, Footer, Modal, Loader
|   |   |-- rider/
|   |   |-- driver/
|   |   `-- admin/
|   |-- pages/
|   |   |-- rider/              // 8 rider pages
|   |   |-- driver/             // 7 driver pages
```

```
|   |   `-- admin/              // 8 admin pages
|   |-- context/
|   |   `-- AuthContext.jsx     // JWT-based auth state
|   |-- services/
|   |   `-- api.js              // Axios instance with interceptors
|   |-- routes/
|   |   `-- AppRoutes.jsx       // React Router configuration
|   |-- App.jsx
|   `-- main.jsx
|-- .env
`-- package.json
```

19.3 Rationale for the Structure

This folder structure directly mirrors the MVC architecture on the backend, with models, controllers, and routes maintained as separate directories to keep responsibilities clearly isolated. On the frontend, pages and components are further sub-divided by user role — Rider, Driver, and Admin — consistent with the application's role-based design, while shared logic such as authentication state and the Axios API client is centralised in the context and services directories respectively to avoid duplication across the 23 React pages.

20. ER Diagram

The Entity-Relationship (ER) diagram below models the five core collections of UCab Premium — Users, Drivers, Cars, Bookings, and Admin — along with the relationships that connect them. Although MongoDB is a schema-flexible, document-oriented database rather than a strictly relational one, the entities in UCab Premium are deliberately modelled with clear, relationally-aware structure using Mongoose schemas and ObjectId references, making a conventional ER representation both valid and useful for understanding the data design.

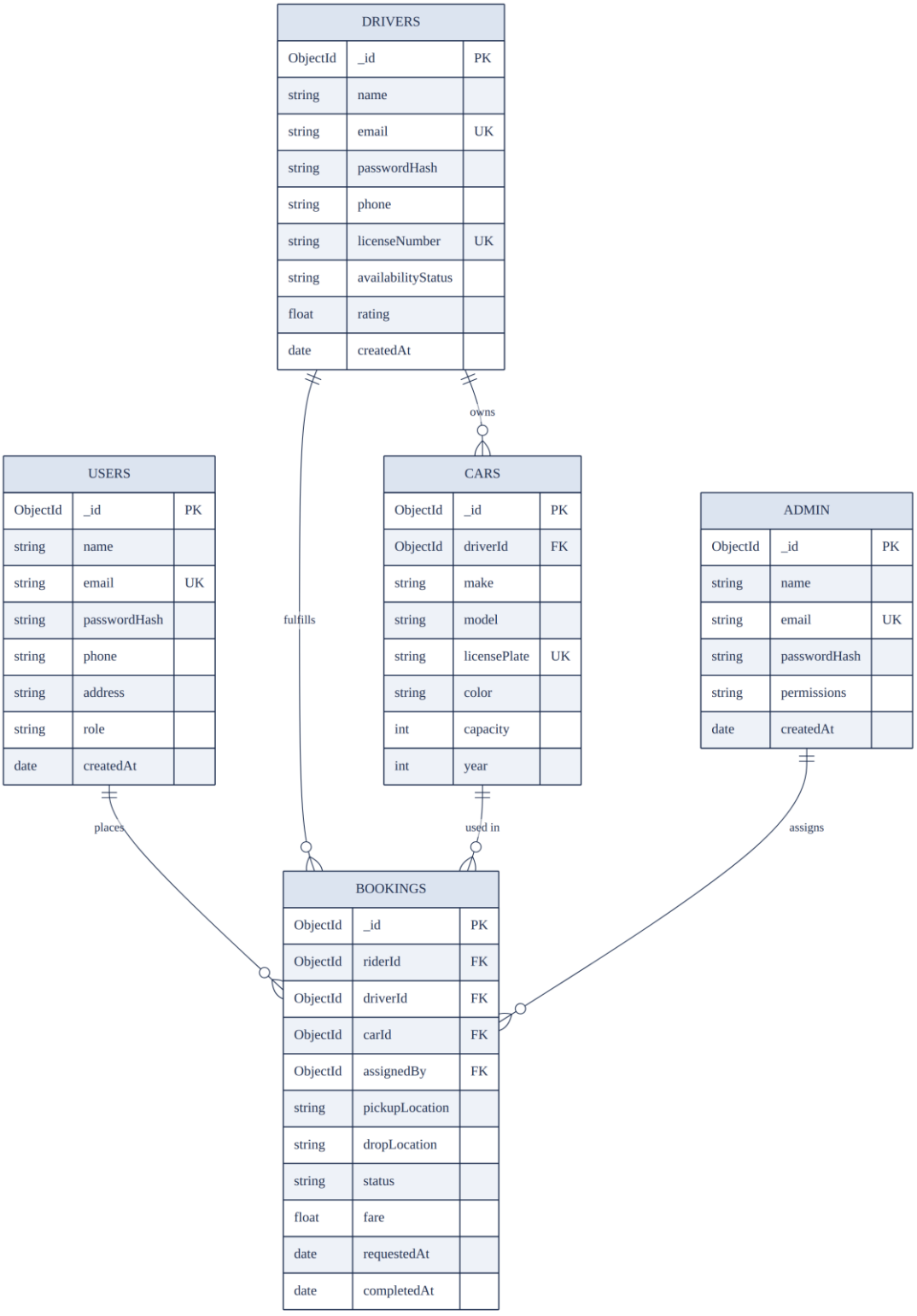

Figure 20.1: Entity-Relationship diagram of the UCab Premium data model

20.1 Explanation of the ER Diagram

The Bookings collection sits at the centre of the data model, as it references three other collections: Users (the rider who placed the booking), Drivers (the driver fulfilling it), and Cars (the vehicle used for the trip). It additionally records which Admin assigned the

driver, linking operational accountability back to the Admin collection. The Users collection is connected to Bookings through a one-to-many relationship, since a single rider may place multiple booking requests over time, but each booking belongs to exactly one rider.

Similarly, the Drivers collection maintains a one-to-many relationship with both Cars and Bookings: a driver may be associated with more than one car record over time (for example, if a vehicle is replaced), and is expected to fulfil many bookings across their time on the platform. The Cars collection, in turn, maintains a one-to-many relationship with Bookings, since a given vehicle may be used across multiple trips. The Admin collection maintains a one-to-many relationship with Bookings through the assignedBy field, reflecting that a single administrator may assign many bookings, while each booking is assigned by exactly one administrator.

21. Database Design

UCab Premium uses MongoDB Atlas, a document-oriented NoSQL database, as its sole data store. Rather than storing data in rigid, table-based rows, MongoDB stores each record as a flexible, JSON-like BSON document. This section explains the key design decisions made while structuring the database for UCab Premium.

21.1 Design Principles

- Each of the five core entities — Users, Drivers, Cars, Bookings, and Admin — is stored in its own dedicated MongoDB collection, mirroring the structure of a normalised relational schema.
- Relationships between collections are implemented using MongoDB ObjectId references rather than embedding, since Bookings, Drivers, and Cars are frequently queried, updated, and read independently of one another.
- Mongoose schemas enforce structural validation — data types, required fields, and enumerated values (such as booking status) — at the application layer, compensating for MongoDB's naturally flexible schema.
- Indexes are applied to frequently queried fields, such as email (Users, Drivers, Admin) and status (Bookings), to improve lookup performance as data volume grows.
- Timestamps (createdAt, updatedAt) are maintained on every collection to support auditing, sorting, and analytics on the admin dashboard.

21.2 Referencing versus Embedding

A key decision in any MongoDB schema design is whether related data should be embedded within a single document or referenced across separate collections. UCab Premium adopts a referencing strategy for its core entities, as summarised in Table 21.1.

Design Approach	When Used in UCab Premium	Reasoning
Referencing (ObjectId)	Bookings referencing Users, Drivers, Cars, and Admin	These entities are large, independently updated, and queried on their own (e.g. a driver's profile is edited without touching every booking they have ever fulfilled).
Embedding	Small, tightly-scoped sub-	Used only where the embedded

Design Approach	When Used in UCab Premium	Reasoning
	documents, e.g. a vehicle's registration details within a Car document	data has no independent existence outside its parent document and is always accessed together with it.

Table 21.1: Referencing versus embedding strategy used across UCab Premium collections

22. MongoDB Collections

UCab Premium's database is organised into five MongoDB collections, each corresponding to a core entity within the system. Table 22.1 summarises the purpose and key indexed fields of each collection.

Collection	Purpose	Key Indexed Field(s)
Users	Stores rider account information used for registration, login, and ride booking.	email (unique)
Drivers	Stores driver account information, license details, and real-time availability status.	email (unique), availabilityStatus
Cars	Stores vehicle details associated with each driver.	licensePlate (unique), driverId
Bookings	Stores every ride request and tracks its progress through the booking lifecycle.	status, riderId, driverId
Admin	Stores administrator accounts used to manage drivers, users, and bookings.	email (unique)

Table 22.1: Overview of MongoDB collections used in UCab Premium

Each collection is defined through a corresponding Mongoose schema on the backend, ensuring that even though MongoDB itself does not enforce a fixed structure, every document written to these collections conforms to a consistent, validated shape.

23. Schema Explanation

This section details the field-level structure of each of the five MongoDB collections, as defined through their respective Mongoose schemas.

23.1 Users Collection

Field	Type	Description
_id	ObjectId	Primary key, automatically generated by MongoDB for each rider document.
name	String	Full name of the rider; required field.
email	String	Rider's email address; required, unique, used as the login identifier.
passwordHash	String	bcryptjs-hashed password; the plain-text password is never stored.
phone	String	Rider's contact phone number, used for booking-related communication.
address	String	Optional default address, used to pre-fill pickup location during booking.
role	String	Fixed value “rider”, used by the authentication middleware for role-based access control.
createdAt	Date	Timestamp automatically recorded when the account is created.

Table 23.1: Field structure of the Users collection

23.2 Drivers Collection

Field	Type	Description
_id	ObjectId	Primary key, automatically generated by MongoDB for each driver document.
name	String	Full name of the driver; required field.
email	String	Driver's email address; required, unique, used as the login identifier.
passwordHash	String	bcryptjs-hashed password for secure driver authentication.
phone	String	Driver's contact phone number.
licenseNumber	String	Driver's official license number; required and unique.
availabilityStatus	String	Enumerated value — Available, Busy, or Offline — governing eligibility for new ride assignments.
rating	Number	Average rider-given rating, used for basic performance

Field	Type	Description
		tracking.
createdAt	Date	Timestamp automatically recorded when the driver account is created.

Table 23.2: Field structure of the Drivers collection

23.3 Cars Collection

Field	Type	Description
_id	ObjectId	Primary key, automatically generated by MongoDB for each car document.
driverId	ObjectId (ref: Drivers)	Foreign-key reference linking the car to its owning driver.
make	String	Manufacturer of the vehicle, e.g. Maruti Suzuki, Hyundai.
model	String	Model name of the vehicle, e.g. Swift Dzire, i20.
licensePlate	String	Government-issued vehicle registration number; required and unique.
color	String	Colour of the vehicle, used for rider identification at pickup.
capacity	Number	Maximum passenger seating capacity of the vehicle.
year	Number	Manufacturing year of the vehicle.

Table 23.3: Field structure of the Cars collection

23.4 Bookings Collection

Field	Type	Description
_id	ObjectId	Primary key, automatically generated by MongoDB for each booking document.
riderId	ObjectId (ref: Users)	Foreign-key reference to the rider who created the booking.
driverId	ObjectId (ref: Drivers)	Foreign-key reference to the driver assigned to the booking; null until assignment.
carId	ObjectId (ref: Cars)	Foreign-key reference to the vehicle used for the trip; populated at assignment.
assignedBy	ObjectId (ref: Admin)	Foreign-key reference to the administrator who assigned the driver.
pickupLocation	String	Rider-specified pickup address for the trip.
dropLocation	String	Rider-specified destination address for the trip.
status	String	Enumerated value representing the current stage of the

Field	Type	Description
		booking lifecycle state machine.
fare	Number	Calculated or fixed trip fare, used for the PDF receipt.
requestedAt	Date	Timestamp recorded when the booking was first created.
completedAt	Date	Timestamp recorded when the trip status changes to Completed; null until then.

Table 23.4: Field structure of the Bookings collection

23.5 Admin Collection

Field	Type	Description
_id	ObjectId	Primary key, automatically generated by MongoDB for each administrator document.
name	String	Full name of the administrator.
email	String	Administrator's email address; required, unique, used as the login identifier.
passwordHash	String	bcryptjs-hashed password for secure administrator authentication.
permissions	String	Permission level of the administrator (e.g. Super Admin, Operations Admin).
createdAt	Date	Timestamp automatically recorded when the administrator account is created.

Table 23.5: Field structure of the Admin collection

24. Relationships Among Collections

Although MongoDB does not enforce foreign-key constraints natively, UCab Premium implements relationships between collections at the application layer using ObjectId references, validated and populated through Mongoose. Table 24.1 summarises each relationship, its cardinality, and how it is implemented.

Relationship	Cardinality	Implementation
Users → Bookings	One-to-Many	Each Booking document stores a riderId referencing the Users collection; a rider may have many bookings.
Drivers → Bookings	One-to-Many	Each Booking document stores a driverId referencing the Drivers collection; a driver may fulfil many bookings.
Drivers → Cars	One-to-Many	Each Car document stores a driverId referencing the Drivers collection; a driver may be linked to multiple car records over time.
Cars → Bookings	One-to-Many	Each Booking document stores a carId referencing the Cars collection; a vehicle may be used across multiple trips.
Admin → Bookings	One-to-Many	Each Booking document stores an assignedBy field referencing the Admin collection; an administrator may assign many bookings.

Table 24.1: Relationships among UCab Premium's MongoDB collections

24.1 Referential Integrity in a NoSQL Context

Since MongoDB does not natively enforce referential integrity the way a relational database would through foreign-key constraints, UCab Premium's backend controllers are responsible for validating references before writing data — for example, confirming that a driverId supplied during ride assignment corresponds to an existing, available driver before updating a Booking document. Mongoose's population feature (.populate()) is used extensively across the API to resolve these references at query time, allowing endpoints such as the admin dashboard to return fully-detailed booking records that include nested rider, driver, and car information without requiring multiple separate client-side requests.

This referencing-based design keeps each collection focused and independently maintainable, while still preserving the relational structure necessary to support complex queries — such as retrieving a driver's complete ride history, or calculating admin dashboard statistics across bookings, drivers, and vehicles.

25. Backend Structure

The backend of UCab Premium is a Node.js and Express.js application that exposes 35 RESTful API endpoints across five resource groups — Authentication, Rider, Driver, Admin, and Booking. The backend is organised following the Model-View-Controller (MVC) architectural pattern, ensuring that data definitions, business logic, and request-handling remain clearly separated across dedicated directories.

At a high level, every incoming request follows a consistent path through the backend: it enters through server.js, is directed by the relevant route file to a middleware chain for authentication and authorisation, and is finally handed to a controller function that executes the required business logic before returning a JSON response. The remainder of this chapter examines each of these components — config, controllers, routes, models, and middleware — in detail.

26. MVC Folder Explanation

Each top-level folder within the server directory corresponds to a specific responsibility within the MVC pattern, as summarised in Table 26.1.

Folder	MVC Role	Contents
config/	Configuration	Database connection setup and environment-level configuration.
models/	Model	Mongoose schemas for Users, Drivers, Cars, Bookings, and Admin.
controllers/	Controller	Business logic and request-handling functions, grouped by resource.
routes/	Routing Layer	Express Router definitions mapping HTTP methods and paths to controller functions.
middleware/	Cross-Cutting Logic	JWT verification and role-based access control, applied before controllers execute.
utils/	Supporting Logic	Helper functions such as PDF receipt generation.

Table 26.1: Mapping of backend folders to MVC responsibilities

This structure ensures that a developer working on, for example, the booking lifecycle state machine, only needs to modify bookingController.js, without needing to touch route definitions, schema validation, or authentication logic — each of which is isolated in its own layer.

27. Config Folder

The config folder contains the setup logic required to connect the Express application to MongoDB Atlas. This is implemented in db.js, which reads the database connection string from an environment variable and establishes the connection using Mongoose when the server starts.

```
// config/db.js
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB Atlas connected successfully');
  } catch (error) {
    console.error('Database connection failed:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Code 27.1: Database connection setup in config/db.js

The MONGO_URI value, along with other sensitive configuration such as the JWT signing secret, is stored in a .env file and loaded using the dotenv package, ensuring that credentials are never hard-coded into the source code or committed to version control.

28. Controllers

Controllers contain the core business logic of UCab Premium. Each controller file groups together the functions responsible for handling requests related to a specific resource — for example, bookingController.js handles all booking-related operations, including creation, assignment, status updates, and history retrieval.

```
// controllers/bookingController.js
const Booking = require('../models/Booking');

exports.createBooking = async (req, res) => {
  try {
    const { pickupLocation, dropLocation } = req.body;
    const booking = await Booking.create({
      riderId: req.user.id,
      pickupLocation,
      dropLocation,
      status: 'Requested',
      requestedAt: new Date(),
    });
    res.status(201).json({ success: true, booking });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};
```

Code 28.1: createBooking function in controllers/bookingController.js

28.1 Sample Request and Response

The example below illustrates the request and response cycle for the createBooking controller function shown above.

```
// Request
POST /api/bookings
Authorization: Bearer <rider_jwt_token>
Content-Type: application/json

{
  "pickupLocation": "Ameerpet, Hyderabad",
  "dropLocation": "Hitech City, Hyderabad"
}

// Response - 201 Created
{
  "success": true,
  "booking": {
    "_id": "665f1a2b3c4d5e6f7a8b9c0d",
    "riderId": "665e...",
    "pickupLocation": "Ameerpet, Hyderabad",
    "dropLocation": "Hitech City, Hyderabad",
```

```
    "status": "Requested",
    "requestedAt": "2026-06-10T09:15:00.000Z"
  }
}
```

Code 28.2: Sample request-response cycle for booking creation

29. Routes

Routes define the mapping between HTTP methods, URL paths, and the controller functions that handle them. Each route file also attaches the appropriate middleware to enforce authentication and role-based access before a request reaches its controller.

```
// routes/bookingRoutes.js
const express = require('express');
const router = express.Router();
const { createBooking, assignDriver, getBookingHistory } =
require('../controllers/bookingController');
const { verifyToken } = require('../middleware/authMiddleware');
const { requireRole } = require('../middleware/roleMiddleware');

router.post('/', verifyToken, requireRole('rider'), createBooking);
router.patch('/:id/assign', verifyToken, requireRole('admin'), assignDriver);
router.get('/history', verifyToken, requireRole('rider'), getBookingHistory);

module.exports = router;
```

Code 29.1: Sample route definitions in routes/bookingRoutes.js

All route files are ultimately mounted onto the main Express application within server.js, each under a distinct base path (e.g. /api/auth, /api/bookings, /api/admin), keeping the 35 endpoints of the API organised into five clearly separated resource groups.

30. Models

Models define the structure, data types, and validation rules for each MongoDB collection using Mongoose schemas. They form the Model layer of the MVC pattern and are the only part of the backend that directly interacts with the database.

```
// models/Booking.js
const mongoose = require('mongoose');

const bookingSchema = new mongoose.Schema({
  riderId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  driverId: { type: mongoose.Schema.Types.ObjectId, ref: 'Driver' },
  carId: { type: mongoose.Schema.Types.ObjectId, ref: 'Car' },
  assignedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'Admin' },
  pickupLocation: { type: String, required: true },
  dropLocation: { type: String, required: true },
  status: {
    type: String,
    enum: ['Requested', 'Accepted', 'Ongoing', 'Completed', 'Cancelled'],
    default: 'Requested',
  },
  fare: { type: Number },
  requestedAt: { type: Date, default: Date.now },
  completedAt: { type: Date },
}, { timestamps: true });

module.exports = mongoose.model('Booking', bookingSchema);
```

Code 30.1: Mongoose schema definition in models/Booking.js

The enum constraint on the status field is particularly important, as it enforces the five valid stages of the booking lifecycle state machine directly at the database layer, preventing an invalid or misspelt status value from ever being persisted.

31. Middleware

Middleware functions execute between the incoming request and the controller that ultimately handles it. UCab Premium uses two primary middleware functions across its protected routes: authMiddleware.js, which verifies the JWT token, and roleMiddleware.js, which restricts access based on the authenticated user's role.

```
// middleware/authMiddleware.js
const jwt = require('jsonwebtoken');

exports.verifyToken = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader) return res.status(401).json({ message: 'No token provided' });

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id, role }
    next();
  } catch (error) {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
};
```

Code 31.1: JWT verification middleware in middleware/authMiddleware.js

```
// middleware/roleMiddleware.js
exports.requireRole = (...allowedRoles) => (req, res, next) => {
  if (!allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Access denied for this role' });
  }
  next();
};
```

Code 31.2: Role-based access control middleware in middleware/roleMiddleware.js

Because verifyToken and requireRole are composed directly within each route definition (as shown in Chapter 29), every protected endpoint enforces both authentication and role-based access control before its controller logic ever executes.

32. JWT Authentication

UCab Premium uses JSON Web Tokens (JWT) to implement stateless authentication across its three user roles. When a user logs in successfully, the server signs a token containing their user ID and role, which the client then attaches to every subsequent request.

```
// controllers/authController.js
const jwt = require('jsonwebtoken');
const bcrypt = require('bcryptjs');
const User = require('../models/User');

exports.login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(404).json({ message: 'User not found' });

  const isMatch = await bcrypt.compare(password, user.passwordHash);
  if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

  const token = jwt.sign(
    { id: user._id, role: 'rider' },
    process.env.JWT_SECRET,
    { expiresIn: '7d' }
  );

  res.status(200).json({ success: true, token, role: 'rider' });
};
```

Code 32.1: Login controller issuing a signed JWT

32.1 Sample Login Request and Response

```
// Request
POST /api/auth/login
Content-Type: application/json

{
  "email": "rider1@ucabpremium.com",
  "password": "SecurePass@123"
}

// Response - 200 OK
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "role": "rider"
}
```

Code 32.2: Sample login request-response cycle

32.2 Token Verification Workflow

On every subsequent request to a protected endpoint, the client attaches the token in the Authorization header using the format Bearer <token>. The verifyToken middleware, described in Chapter 31, decodes and validates this token using the same JWT_SECRET used to sign it. If valid, the decoded payload — containing the user's ID and role — is attached to req.user, making it available to the controller for authorisation checks and database queries. If the token is missing, malformed, or expired, the middleware immediately returns a 401 Unauthorized response, preventing the request from reaching any controller logic.

32.3 Additional Security Practices

Beyond password hashing and JWT-based authentication, UCab Premium applies several complementary security practices that are standard for production-grade REST APIs. These measures are summarised below and are consistent with the OWASP guidelines for securing Node.js and Express applications.

- **Environment Variable Management:** All secrets — including JWT_SECRET, the MongoDB Atlas connection string, and the token expiry window — are stored in a .env file loaded via the dotenv package and excluded from version control through .gitignore, preventing credential leakage through the source repository.
- **Input Validation:** Every request body is validated at the controller layer before it reaches business logic or the database, rejecting malformed payloads (missing fields, invalid email formats, out-of-range values) with a 400 Bad Request response rather than allowing unchecked data to propagate into Mongoose schemas.
- **CORS Configuration:** Cross-Origin Resource Sharing (CORS) is configured on the Express server to accept requests only from the deployed frontend origin, preventing unauthorised third-party domains from making authenticated calls to the API using a victim’s stored credentials.
- **HTTPS Enforcement:** The production deployment is served exclusively over HTTPS, ensuring that JWTs, credentials, and booking data are encrypted in transit and cannot be intercepted through man-in-the-middle attacks on the network path between client and server.
- **Defence in Depth:** Combined with the role-based access control described in Chapter 9, these measures ensure that authentication tokens, stored credentials, and

API traffic are protected at every layer of the request lifecycle — from the browser, across the network, to the database.

Table 32.1 consolidates these measures alongside the threats they mitigate.

Security Measure	Threat Mitigated
Password Hashing (bcryptjs)	Database compromise exposing reusable plain-text credentials.
JWT Signing and Expiry	Session hijacking and indefinite token reuse after theft.
Environment Variable Isolation	Credential leakage through source control or public repositories.
Server-Side Input Validation	Injection attacks and malformed data corrupting the database.
CORS Restriction	Cross-origin requests from unauthorised third-party domains.
HTTPS Enforcement	Interception of credentials and tokens via man-in-the-middle attacks.

Table 32.1: Security measures implemented in UCab Premium and the threats they mitigate

33. Password Encryption Using bcryptjs

To protect user credentials, UCab Premium never stores plain-text passwords. Instead, passwords are hashed using bcryptjs at the point of registration, and the resulting hash — not the original password — is what gets saved to the corresponding MongoDB collection.

```
// controllers/authController.js
const bcrypt = require('bcryptjs');
const User = require('../models/User');

exports.register = async (req, res) => {
  const { name, email, password, phone } = req.body;

  const salt = await bcrypt.genSalt(10);
  const passwordHash = await bcrypt.hash(password, salt);

  const user = await User.create({ name, email, passwordHash, phone, role: 'rider' });
  res.status(201).json({ success: true, userId: user._id });
};
```

Code 33.1: Password hashing during registration using bcryptjs

33.1 How bcryptjs Protects Passwords

- genSalt(10) generates a random salt with a work factor of 10, meaning the hashing algorithm runs 2^10 internal rounds, deliberately slowing down brute-force attempts.
- Each password is hashed with a unique, randomly generated salt, so two users with an identical password will still have completely different stored hashes.
- bcrypt.compare() is used at login (as shown in Code 32.1) to check a submitted password against the stored hash without ever needing to decrypt or reveal the original password.
- Because bcrypt hashing is intentionally one-way, even a full database compromise would not directly expose usable plain-text passwords.

Aspect	Description
Algorithm	bcrypt — a salted, adaptive hashing function designed specifically for password storage.
Salt Rounds	10 (used consistently across Users, Drivers, and Admin registration flows).
Stored Value	passwordHash field — the plain-text password itself is never persisted.

Aspect	Description
Verification Method	bcrypt.compare(plainPassword, storedHash), used during login for all three roles.

Table 33.1: Summary of bcryptjs password protection in UCab Premium

34. API Documentation

UCab Premium exposes 35 RESTful API endpoints, organised into five resource groups. Table 34.1 documents the primary endpoints within each group, including their HTTP method, path, purpose, and authentication requirement.

34.1 Authentication Endpoints

Method	Endpoint	Description	Auth Required
POST	`/api/auth/register`	Registers a new Rider, Driver, or Admin account.	No
POST	`/api/auth/login`	Authenticates a user and issues a signed JWT.	No
GET	`/api/auth/me`	Returns the profile of the currently authenticated user.	Yes (Any role)

Table 34.1: Authentication resource group endpoints

34.2 Rider Endpoints

Method	Endpoint	Description	Auth Required
POST	`/api/bookings`	Creates a new ride booking request.	Yes (Rider)
GET	`/api/bookings/history`	Retrieves the rider's complete booking history.	Yes (Rider)
GET	`/api/bookings/:id/receipt`	Downloads the PDF receipt for a completed booking.	Yes (Rider)
PATCH	`/api/bookings/:id/cancel`	Cancels a pending booking.	Yes (Rider)

Table 34.2: Rider resource group endpoints

34.3 Driver Endpoints

Method	Endpoint	Description	Auth Required
PATCH	`/api/drivers/availability`	Toggles the driver's availability status.	Yes (Driver)
PATCH	`/api/bookings/:id/start`	Marks an assigned booking as Ongoing.	Yes (Driver)
PATCH	`/api/bookings/:id/complete`	Marks a booking as Completed and triggers receipt generation.	Yes (Driver)
GET	`/api/drivers/bookings`	Retrieves bookings assigned to the authenticated driver.	Yes (Driver)

Table 34.3: Driver resource group endpoints

34.4 Admin Endpoints

Method	Endpoint	Description	Auth Required
GET	`/api/admin/dashboard`	Returns the 10 statistical summary cards and analytics data.	Yes (Admin)
PATCH	`/api/bookings/:id/assign`	Assigns an available driver to a pending booking.	Yes (Admin)
GET	`/api/admin/users`	Retrieves a searchable, filterable list of registered riders.	Yes (Admin)
GET	`/api/admin/drivers`	Retrieves a searchable, filterable list of registered drivers.	Yes (Admin)

Table 34.4: Admin resource group endpoints

34.5 Booking Lifecycle Endpoints

Method	Endpoint	Description	Auth Required
GET	`/api/bookings/:id`	Retrieves full details of a single booking, with populated rider, driver, and car data.	Yes (Any role)
GET	`/api/bookings`	Retrieves all bookings, filterable by status.	Yes (Admin)

Table 34.5: Booking resource group endpoints

34.6 Standard Response Format

All API responses across the 35 endpoints follow a consistent JSON structure, indicating success or failure along with the relevant payload or error message, as shown below.

```
// Success response format
{
  "success": true,
  "data": { /* resource-specific payload */ }
}

// Error response format
{
  "success": false,
  "message": "Descriptive error message"
}
```

Code 34.1: Standard success and error response formats used across the API

35. React Project Structure

The UCab Premium frontend is a React single-page application (SPA) comprising 23 pages organised across three role-specific modules — Rider, Driver, and Admin. The project follows a modular, feature-oriented structure that separates reusable UI components, page-level views, shared state, and API communication logic into distinct directories, as introduced in Chapter 19.

Directory	Purpose
src/components/	Reusable, presentation-focused UI building blocks used across multiple pages.
src/pages/	Full page-level views, one per application route, organised into rider/, driver/, and admin/ sub-folders.
src/context/	React Context providers for global state, most notably authentication state.
src/services/	Centralised Axios instance and API call functions.
src/routes/	React Router configuration, including protected and role-based route definitions.
src/assets/	Static assets such as icons, logos, and images used throughout the interface.

Table 35.1: Purpose of each top-level directory within the React frontend

This structure keeps concerns cleanly separated: a component in src/components/ never fetches data directly, a page in src/pages/ never defines raw Axios calls inline, and authentication state is never duplicated locally within individual components — it is always read from the shared AuthContext, described later in this chapter.

36. Components

Components are the smallest reusable building blocks of the UCab Premium interface. Each component is designed to be presentation-focused, receiving data and callback functions through props rather than fetching data itself, which keeps components easily reusable across the Rider, Driver, and Admin modules.

Component	Location	Description
Navbar	components/common/	Top navigation bar, dynamically rendering links based on the authenticated user's role.
Footer	components/common/	Site-wide footer displayed across all pages.
Modal	components/common/	Reusable modal dialog used for confirmations, such as booking cancellation or driver assignment.
Loader / Skeleton	components/common/	Skeleton loading placeholders shown while dashboard or list data is being fetched.
StatCard	components/admin/	Displays a single statistic (e.g. Total Bookings, Active Drivers) on the admin dashboard's 10-card summary grid.
BookingCard	components/rider/	Displays a summary of a single booking within the rider's ride history list.
DriverAvailabilityToggle	components/driver/	Switch control allowing a driver to toggle between Available and Offline.
AnalyticsChart	components/admin/	Wraps Recharts components to render booking and revenue trend charts on the admin dashboard.

Table 36.1: Key reusable components in the UCab Premium frontend

```
// components/rider/BookingCard.jsx
import React from 'react';

function BookingCard({ booking }) {
  return (
    <div className="booking-card">
      <p className="route">{booking.pickupLocation} → {booking.dropLocation}</p>
      <span className={`status-badge status-${booking.status.toLowerCase()}`}>
        {booking.status}
      </span>
      <p className="fare">₹{booking.fare ?? '--'}</p>
    </div>
  );
}

export default BookingCard;
```

Code 36.1: BookingCard component, receiving booking data as a prop

As shown in Code 36.1, BookingCard is a purely presentational component — it receives a booking object as a prop and renders it, without containing any API calls or state logic of its own. This pattern is followed consistently across UCab Premium's components, keeping data-fetching responsibility isolated to the page level.

37. Pages

Pages represent complete, routable views of the application and are responsible for fetching the data they need — typically through the Axios service layer described in Chapter 39 — and composing it using the reusable components from Chapter 36. UCab Premium's 23 pages are distributed across the three role-specific modules as shown in Table 37.1.

Module	Page Count	Representative Pages
Rider	8	Login, Register, BookRide, RideHistory, RideDetails, Profile, ReceiptView, RiderDashboard
Driver	7	Login, DriverDashboard, AssignedRides, RideInProgress, AvailabilityToggle, RideHistory, Profile
Admin	8	Login, AdminDashboard, ManageUsers, ManageDrivers, ManageBookings, Analytics, AssignDriver, AdminProfile

Table 37.1: Distribution of the 23 React pages across the Rider, Driver, and Admin modules

```
// pages/rider/RideHistory.jsx
import React, { useEffect, useState } from 'react';
import api from '../../services/api';
import BookingCard from '../../components/rider/BookingCard';
import Loader from '../../components/common/Loader';

function RideHistory() {
  const [bookings, setBookings] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get('/bookings/history')
      .then((res) => setBookings(res.data.data))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <Loader />;

  return (
    <div className="ride-history">
      {bookings.map((b) => <BookingCard key={b._id} booking={b} />)}
    </div>
  );
}

export default RideHistory;
```

Code 37.1: RideHistory page fetching data and composing BookingCard components

This page-level pattern — fetch on mount using useEffect, track a loading state, and render either a skeleton Loader or the composed list of components — is repeated consistently across all 23 pages, giving the application a predictable, maintainable data-fetching workflow.

38. Routing

Client-side navigation across UCab Premium's 23 pages is managed using React Router, which allows the application to switch between views without triggering a full page reload, preserving the single-page application experience. Routes are centrally defined in src/routes/AppRoutes.jsx.

```
// routes/AppRoutes.jsx
import { Routes, Route } from 'react-router-dom';
import Login from '../pages/rider/Login';
import BookRide from '../pages/rider/BookRide';
import RideHistory from '../pages/rider/RideHistory';
import AdminDashboard from '../pages/admin/AdminDashboard';
import ProtectedRoute from './ProtectedRoute';

function AppRoutes() {
  return (
    <Routes>
      <Route path="/login" element={<Login />} />

      <Route path="/book" element={
        <ProtectedRoute allowedRoles={['rider']}><BookRide /></ProtectedRoute>
      } />

      <Route path="/rides/history" element={
        <ProtectedRoute allowedRoles={['rider']}><RideHistory /></ProtectedRoute>
      } />

      <Route path="/admin/dashboard" element={
        <ProtectedRoute allowedRoles={['admin']}><AdminDashboard /></ProtectedRoute>
      } />
    </Routes>
  );
}

export default AppRoutes;
```

Code 38.1: Centralised route definitions in routes/AppRoutes.jsx

Each protected route is wrapped in the ProtectedRoute component, which is described fully in Chapter 40. This ensures that routing and access control are handled consistently in a single, centralised location rather than being scattered across individual page components.

39. Axios API Integration

All communication between the React frontend and the Express backend is handled through a single, centrally configured Axios instance defined in src/services/api.js. This instance automatically attaches the JWT token to every outgoing request using an Axios request interceptor, so individual pages never need to manually manage authentication headers.

```
// services/api.js
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
});

// Attach JWT token to every outgoing request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Handle expired/invalid tokens globally
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

Code 39.1: Centralised Axios instance with request and response interceptors

The response interceptor shown above provides a consistent, application-wide handling of expired or invalid JWT tokens: whenever the backend returns a 401 Unauthorized status, the interceptor automatically clears the stored token and redirects the user to the login page, ensuring that session expiry is handled gracefully and consistently regardless of which page triggered the failing request.

40. Protected Routes

Protected routes ensure that a given page can only be accessed by users who are both authenticated and hold the correct role. This is implemented through a wrapper component, ProtectedRoute, which reads the current authentication state from AuthContext and either renders the requested page or redirects the user.

```
// routes/ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

function ProtectedRoute({ children, allowedRoles }) {
  const { user, isAuthenticated } = useAuth();

  if (!isAuthenticated) return <Navigate to="/login" replace />;

  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/unauthorized" replace />;
  }

  return children;
}

export default ProtectedRoute;
```

Code 40.1: ProtectedRoute component enforcing authentication and role checks

40.1 Protected Route Workflow

- If the user is not authenticated, ProtectedRoute redirects them to /login before the requested page ever renders.
- If the user is authenticated but their role is not included in allowedRoles, they are redirected to an /unauthorized page rather than being shown the restricted content.
- Only when both checks pass does ProtectedRoute render its children — the actual page component — mirroring the role-based access control enforced independently on the backend (Chapter 31).

This client-side protection improves user experience by preventing unauthorised users from ever seeing a restricted page's layout, but it is treated strictly as a convenience layer: the backend's JWT verification and role middleware remain the actual source of enforced security, since client-side checks alone can always be bypassed.

41. Forms

Forms throughout UCab Premium — including login, registration, and ride booking — are implemented as controlled components, meaning that each input field's value is driven by React state rather than the DOM itself, allowing for real-time validation and consistent form behaviour.

```
// pages/rider/BookRide.jsx
import React, { useState } from 'react';
import api from '../../services/api';

function BookRide() {
  const [form, setForm] = useState({ pickupLocation: '', dropLocation: '' });
  const [error, setError] = useState('');

  const handleChange = (e) =>
    setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!form.pickupLocation || !form.dropLocation) {
      setError('Both pickup and drop locations are required.');
      return;
    }
    await api.post('/bookings', form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="pickupLocation" value={form.pickupLocation} onChange={handleChange} />
      <input name="dropLocation" value={form.dropLocation} onChange={handleChange} />
      {error && <p className="form-error">{error}</p>}
      <button type="submit">Book Ride</button>
    </form>
  );
}

export default BookRide;
```

Code 41.1: Controlled booking form with client-side validation in BookRide.jsx

Basic client-side validation, such as checking for empty required fields, is performed before a request is sent to the API, improving responsiveness and reducing unnecessary network calls. However, all fields are still re-validated on the backend by the corresponding Mongoose schema, ensuring that data integrity does not depend solely on frontend logic.

42. State Management

UCab Premium uses a combination of local component state and global state to manage data across its 23 pages. Local, page-specific data (such as form inputs or fetched lists) is managed with React's built-in useState and useEffect hooks, while state that must be shared across many components — most notably authentication state — is managed globally using React's Context API.

```
// context/AuthContext.jsx
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const stored = localStorage.getItem('user');
    return stored ? JSON.parse(stored) : null;
  });

  const login = (userData, token) => {
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(userData));
    setUser(userData);
  };

  const logout = () => {
    localStorage.clear();
    setUser(null);
  };

  const value = { user, isAuthenticated: !!user, login, logout };
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => useContext(AuthContext);
```

Code 42.1: Global authentication state managed via React Context in AuthContext.jsx

State Type	Tool Used	Example Usage
Local UI state	useState	Form inputs, modal open/close, loading flags.
Derived/side-effect state	useEffect	Fetching data from the API when a page mounts.
Global authentication state	React Context (AuthContext)	Current user, role, and login/logout functions, accessible from any component via useAuth().

Table 42.1: State management approaches used across the UCab Premium frontend

This layered approach avoids the added complexity of a dedicated state management library such as Redux, which was not necessary given the application's scope: authentication is the only truly global state requirement, while all other state remains naturally scoped to the page or component that owns it.

43. User Interface Design

UCab Premium's interface follows a premium, cohesive design language built around a dark navy and amber colour palette, intended to convey a professional, trustworthy aesthetic appropriate for a transportation platform. This design system is applied consistently across all 23 pages and reusable components.

Design Element	Description
Colour Palette	Dark navy (#1A2B4C) as the primary background/brand colour, paired with amber/gold (#B8860B) accents for calls-to-action and highlights.
Typography	A clean, modern sans-serif type system used for UI text, ensuring readability across dashboards and data-dense admin views.
Animations	Subtle transition and fade-in animations applied to page loads, modals, and status changes to give the interface a polished feel.
Skeleton Loaders	Placeholder loading states shown while dashboard statistics, charts, or lists are being fetched, replacing generic spinners.
Modals	Used for confirmation actions such as ride cancellation or driver assignment, keeping the user within context rather than navigating away.
Status Badges	Colour-coded badges (e.g. distinct colours for Requested, Ongoing, Completed) used throughout booking-related views for quick visual scanning.

Table 43.1: Key elements of the UCab Premium design system

43.1 Design Consistency Across Roles

While the Rider, Driver, and Admin modules serve different functional purposes, all three share the same underlying design system — colour palette, component styling, and interaction patterns — implemented through shared CSS and common components such as Navbar, Modal, and Loader. This ensures that a user switching context (for example, an administrator who is also a registered rider) experiences a visually consistent product, while each role's dashboard is tailored to surface the specific data and actions relevant to that role, such as the Admin module's 10-card statistics grid and Recharts-based analytics visualisations.

44. User Module

The User Module (Rider Module) governs every interaction available to a Rider on UCab Premium, from account creation through to booking rides and reviewing past trips. It is the primary consumer-facing module of the platform.

Feature	Description
Registration	Allows a new rider to create an account with name, email, phone, and password.
Login	Authenticates a rider and issues a JWT token scoped to the rider role.
Book a Ride	Allows a rider to submit a new booking request with pickup and drop locations.
Ride History	Displays all past and current bookings placed by the rider, with status indicators.
PDF Receipt Download	Allows a rider to download a formal PDF receipt for any completed ride.
Profile Management	Allows a rider to view and update their personal account details.

Table 44.1: Features available within the User Module

44.1 Step-by-Step Flow: Rider Registration and Login

1. The rider navigates to the registration page and submits their name, email, phone number, and password.
2. The backend validates the submitted data, hashes the password using bcryptjs, and creates a new document in the Users collection.
3. The rider is redirected to the login page, where they submit their registered email and password.
4. The backend verifies the credentials using bcrypt.compare() and, if valid, issues a signed JWT token containing the rider's ID and role.
5. The frontend stores the token and redirects the rider to their dashboard, from which they can book a ride or view their ride history.

45. Admin Module

The Admin Module provides the operational control centre of UCab Premium, giving administrators the tools needed to oversee riders, drivers, and bookings across the platform, and to monitor overall system performance through analytics.

Feature	Description
Admin Dashboard	Displays 10 statistical summary cards and Recharts-based analytical visualisations of platform activity.
Driver Assignment	Allows an administrator to assign a pending booking to any driver currently marked as Available.
User Management	Allows searching, filtering, and reviewing registered rider accounts.
Driver Management	Allows searching, filtering, and reviewing registered driver accounts and their availability status.
Booking Oversight	Allows administrators to view all bookings across the platform, filterable by status.
Search and Filter Tools	Provides quick lookup of users, drivers, and bookings across admin pages.

Table 45.1: Features available within the Admin Module

45.1 Step-by-Step Flow: Admin Monitoring and Assignment

6. The administrator logs in and is redirected to the Admin Dashboard, displaying real-time statistics and charts.
7. The administrator navigates to the pending bookings list, filtered to show bookings with a Requested status.

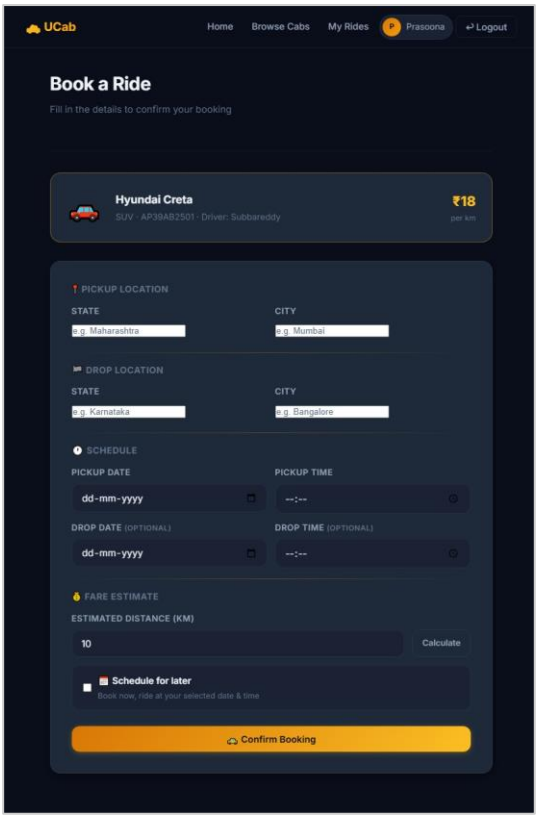

Figure 57.6: Book a Ride screen

8. The administrator selects a booking and reviews the list of currently available drivers.
9. The administrator assigns a driver to the booking, triggering the Driver Assignment Module described in Chapter 50.
10. The dashboard statistics update to reflect the newly assigned booking, providing the administrator with continuous operational visibility.

46. Driver Module

The Driver Module governs every interaction available to a Driver, including account management, availability control, and the handling of assigned rides through to completion.

Feature	Description
Registration	Allows a new driver to register with their name, email, phone, and license number.
Login	Authenticates a driver and issues a JWT token scoped to the driver role.
Availability Toggle	Allows a driver to switch between Available and Offline, controlling their eligibility for new assignments.
Assigned Ride Management	Allows a driver to view, start, and complete rides assigned to them by an administrator.
Ride History	Displays a driver's complete history of fulfilled bookings.
Profile Management	Allows a driver to view and update their personal and license details.

Table 46.1: Features available within the Driver Module

46.1 Step-by-Step Flow: Driver Going Online and Fulfilling a Ride

11. The driver logs in and is redirected to the Driver Dashboard.
12. The driver toggles their availability status from Offline to Available, making them eligible for new ride assignments.
13. Once an administrator assigns a booking to the driver, the driver's availability status automatically changes to Busy, and the booking appears in their assigned rides list.
14. The driver starts the trip from their dashboard, transitioning the booking to the Ongoing status.
15. Upon reaching the destination, the driver marks the ride as complete, triggering the Ride Completion Workflow described in Chapter 52, after which their availability automatically reverts to Available.

47. Car Module

The Car Module manages the vehicle records associated with each driver. Every driver must have at least one registered car before they can be assigned a booking, ensuring that vehicle details are always available to riders and administrators.

Feature	Description
Car Registration	Allows a driver to add a vehicle's make, model, license plate, colour, and seating capacity.
Car-Driver Association	Links each car record to its owning driver via a driverId reference.
Car Details Display	Surfaces vehicle details (make, model, colour, plate) to riders once a driver is assigned to their booking.
Car Record Update	Allows a driver to update or replace their registered vehicle details.

Table 47.1: Features available within the Car Module

47.1 Step-by-Step Flow: Registering a Vehicle

16. During or after driver registration, the driver navigates to the vehicle details section of their profile.
17. The driver submits the vehicle's make, model, license plate, colour, capacity, and year of manufacture.
18. The backend validates the submitted details and creates a new document in the Cars collection, linked to the driver's ID.
19. This car record becomes available for reference whenever the driver is subsequently assigned to a booking, allowing the rider to view vehicle details ahead of pickup.

48. Booking Module

The Booking Module is the functional core of UCab Premium, coordinating the creation, assignment, progress, and closure of every ride request placed on the platform. It draws together the User, Driver, Car, and Admin modules into a single, cohesive process.

Feature	Description
Create Booking	Allows a rider to submit a new ride request with pickup and drop locations.
Cancel Booking	Allows a rider or administrator to cancel a booking prior to completion.
Status Tracking	Maintains the current stage of each booking through the five-stage lifecycle state machine.
Fare Recording	Stores the calculated or fixed fare for a completed booking.
Receipt Generation	Automatically generates a PDF receipt once a booking reaches the Completed status.

Table 48.1: Features available within the Booking Module

The detailed end-to-end behaviour of the Booking Module is described further in Chapter 51 (Booking Workflow) and Chapter 52 (Ride Completion Workflow), which trace a booking's full journey from creation to closure.

49. Authentication Module

The Authentication Module underpins every other module in UCab Premium, providing secure identity verification and role-based access control for Riders, Drivers, and Administrators, as introduced technically in Chapters 32 and 33.

Feature	Description
Registration	Creates a new account with a securely hashed password, for any of the three user roles.
Login	Verifies credentials and issues a signed JWT token scoped to the user's role.
Token Verification	Validates the JWT on every protected request via the authMiddleware.
Role-Based Access Control	Restricts access to specific endpoints and pages based on the role embedded in the JWT.
Session Expiry Handling	Automatically logs out and redirects the user when their token expires or becomes invalid.

Table 49.1: Features available within the Authentication Module

49.1 Step-by-Step Flow: Authentication Lifecycle

20. A user (Rider, Driver, or Admin) submits their email and password through the login form.
21. The backend locates the matching account and uses bcrypt.compare() to verify the submitted password against the stored hash.
22. Upon successful verification, the server signs a JWT containing the user's ID and role, and returns it to the client.
23. The frontend stores the token and attaches it to the Authorization header of every subsequent API request via the Axios interceptor.
24. On each protected request, the authMiddleware verifies the token's signature and expiry before allowing the request to reach its controller; the roleMiddleware then confirms the user's role is permitted for that specific endpoint.
25. If the token expires or is rejected at any point, the response interceptor clears the stored session and redirects the user to the login page.

50. Driver Assignment Module

The Driver Assignment Module governs how a pending booking is matched with a suitable driver. This process is deliberately kept under administrator control rather than fully automated, allowing for human oversight of the assignment while still enforcing strict availability rules.

Feature	Description
Available Driver Listing	Displays only drivers whose availabilityStatus is currently Available.
Manual Assignment	Allows an administrator to select and assign a specific available driver to a pending booking.
Availability Enforcement	Prevents assignment of any driver who is not currently marked as Available, avoiding double-booking.
Automatic Status Update	Updates both the booking status and the assigned driver's availability status as a single, coordinated operation.

Table 50.1: Features available within the Driver Assignment Module

50.1 Step-by-Step Flow: Assigning a Driver to a Booking

26. The administrator selects a booking currently in the Requested status from the pending bookings list.
27. The system queries the Drivers collection for all documents where availabilityStatus equals “Available”.
28. The administrator selects a driver from this filtered list and confirms the assignment.
29. The backend performs two coordinated updates: the Booking document's status changes to Accepted and its driverId and carId fields are populated, while the Driver document's availabilityStatus changes to Busy.
30. Because the driver list was restricted to Available drivers only, this process structurally prevents the same driver from being assigned to two overlapping bookings.

51. Booking Workflow

The Booking Workflow traces the complete journey of a ride request from the moment a rider submits it through to driver assignment and the start of the trip, governed throughout by the five-stage booking lifecycle state machine.

31. Requested: The rider submits a booking with pickup and drop locations. A new Booking document is created with status Requested, and the booking appears in the administrator's pending bookings queue.
32. Reviewed: The administrator reviews the pending booking alongside the current list of available drivers.
33. Accepted: The administrator assigns an available driver to the booking. The booking's status changes to Accepted, and the assigned driver's availability changes to Busy, as detailed in Chapter 50.
34. Notified: The assigned driver sees the new ride in their dashboard's assigned rides list, along with the rider's pickup and drop details.
35. Ongoing: When the driver begins the trip, the booking's status changes to Ongoing, reflecting that the ride is actively in progress.
36. Cancellation Path (Alternative): At any point prior to completion, the rider or an administrator may cancel the booking, immediately setting its status to Cancelled and, if a driver had already been assigned, reverting that driver's availability back to Available.

This workflow ensures that every booking's state accurately reflects its real-world progress at all times, and that the transition between each stage is explicitly validated by the backend rather than inferred from client-side assumptions.

52. Ride Completion Workflow

The Ride Completion Workflow describes the final stage of a booking's lifecycle, covering the transition from an Ongoing trip to a fully closed, receipted booking.

37. Trip Completion: Upon reaching the drop location, the driver marks the ride as complete through their dashboard.
38. Status Update: The backend updates the Booking document's status to Completed and records the completedAt timestamp.
39. Driver Availability Reset: The assigned driver's availabilityStatus is automatically reverted from Busy back to Available, making them eligible for new assignments immediately.
40. Fare Finalisation: The final fare for the trip is recorded on the Booking document.
41. Receipt Generation: The backend invokes the PDFKit-based receipt service to generate a formal PDF receipt summarising the trip's pickup and drop locations, driver and vehicle details, fare, and timestamps.
42. Rider Notification: The rider's interface reflects the updated Completed status, and the generated receipt becomes available for download from their ride history.
43. Analytics Update: The completed booking is factored into the Admin Dashboard's statistical summary cards and analytics charts, ensuring administrators always have an up-to-date view of platform activity.

This workflow closes the loop initiated in Chapter 51, ensuring that every booking that reaches completion leaves behind an accurate record, a formal receipt, and an immediately available driver — reinforcing the reliability and operational discipline that UCab Premium was designed to provide.

53. Implementation Details

UCab Premium was implemented in two structured development phases over the internship duration of May 2026 to July 2026. Phase 1 established the core booking lifecycle, driver availability state management, and administrator-controlled driver assignment. Phase 2 built on this foundation with search and filter tools, an expanded ten-card analytics dashboard, Recharts-based visualisations, automated PDF receipt generation, and a complete premium dark navy and amber UI redesign.

Aspect	Implementation Approach
Development Methodology	Iterative, phase-based development, with Phase 1 delivering core functionality and Phase 2 layering on analytics and UI refinement.
Coding Standards	Consistent MVC-based file organisation, camelCase naming for variables/functions, PascalCase for React components, and RESTful conventions for all API routes.
Version Control	Git, with logically scoped commits and a dedicated GitHub repository (ucab-mern-booking-system) including a professional README and MIT License.
Environment Configuration	Sensitive configuration (JWT secret, MongoDB URI) managed through .env files, excluded from version control via .gitignore.
Code Reuse	Shared reusable components (Navbar, Modal, Loader, StatCard) and a centralised Axios instance to avoid duplication across the 23 React pages.
API Design	35 RESTful endpoints across five resource groups, each following a consistent JSON request/response structure.

Table 53.1: Summary of implementation practices followed in UCab Premium

54. Project Execution Steps

The following steps outline the practical sequence in which UCab Premium was planned, built, and refined over the course of the internship.

44. Requirement Gathering: Defined the functional and non-functional requirements for the Rider, Driver, and Admin modules, as documented in Chapters 9 and 10.
45. Database Design: Modelled the five MongoDB collections — Users, Drivers, Cars, Bookings, and Admin — and defined their relationships, as detailed in Chapters 20 to 24.
46. Backend Development (Phase 1): Implemented the Express server, Mongoose models, JWT authentication, and the core booking lifecycle state machine and driver availability logic.
47. Frontend Development (Phase 1): Built the initial React pages for rider booking, driver dashboards, and admin driver assignment, connected to the backend via Axios.
48. Integration and Initial Testing: Connected the frontend and backend end-to-end, verifying that booking creation, assignment, and status updates worked correctly across roles.
49. Backend Enhancement (Phase 2): Added search and filter endpoints, the ten-card analytics aggregation logic, and the PDFKit-based receipt generation service.
50. Frontend Enhancement (Phase 2): Implemented the Recharts analytics dashboard, PDF receipt download flow, and the full dark navy and amber UI redesign with animations and skeleton loaders.
51. Testing: Conducted unit, integration, and API-level testing across both backend and frontend, as detailed in Chapter 55.
52. Documentation: Produced formal, university-standard project documentation covering all architectural, functional, and implementation aspects of the system.

55. Testing

UCab Premium was tested across three complementary levels — unit testing, integration testing, and API testing — to verify correctness at the level of individual functions, the interaction between backend components, and the behaviour of the exposed REST API as a whole.

55.1 Unit Testing

Unit tests verify the smallest testable pieces of the application in isolation, such as individual controller functions or utility logic, independent of the database or network layer.

```
// tests/unit/bookingStatus.test.js
const { isValidTransition } = require('../../utils/bookingStateMachine');

test('allows transition from Requested to Accepted', () => {
  expect(isValidTransition('Requested', 'Accepted')).toBe(true);
});

test('rejects transition from Requested directly to Completed', () => {
  expect(isValidTransition('Requested', 'Completed')).toBe(false);
});
```

Code 55.1: Sample unit test verifying booking lifecycle state transitions

Unit tests such as the one above were used to validate the rules of the booking lifecycle state machine, JWT token generation logic, and bcryptjs password hashing and comparison functions, ensuring each behaved correctly in isolation before being exercised through the full API.

55.2 Integration Testing

Integration tests verify that multiple components — typically a controller, its Mongoose model, and the MongoDB database — work correctly together, catching issues that unit tests alone cannot detect, such as incorrect population of referenced documents.

```
// tests/integration/booking.test.js
test('creates a booking and links it to the correct rider', async () => {
  const res = await request(app)
    .post('/api/bookings')
    .set('Authorization', `Bearer ${riderToken}`)
    .send({ pickupLocation: 'Ameerpet', dropLocation: 'Hitech City' });

  expect(res.statusCode).toBe(201);
```

```
  expect(res.body.booking.status).toBe('Requested');
  expect(res.body.booking.riderId).toBe(riderId);
});
```

Code 55.2: Sample integration test for the booking creation endpoint

Integration testing was particularly important for verifying the Driver Assignment Module, confirming that assigning a driver correctly and atomically updated both the Booking document's status and the Driver document's availability in a single, coordinated operation.

55.3 API Testing

API-level testing was carried out using Postman throughout development, verifying that each of the 35 REST endpoints returned the correct status codes, response structure, and authentication behaviour under both valid and invalid conditions.

Test Case	Endpoint	Expected Result
Login with valid credentials	`POST /api/auth/login`	200 OK with a valid JWT token returned.
Login with incorrect password	`POST /api/auth/login`	401 Unauthorized with an appropriate error message.
Create booking without a token	`POST /api/bookings`	401 Unauthorized, request rejected by authMiddleware.
Assign driver as a non-admin user	`PATCH /api/bookings/:id/assign`	403 Forbidden, rejected by roleMiddleware.
Assign an already-busy driver	`PATCH /api/bookings/:id/assign`	400 Bad Request, rejected by assignment validation logic.
Fetch admin dashboard statistics	`GET /api/admin/dashboard`	200 OK with all 10 statistic fields populated.

Table 55.1: Sample API test cases executed via Postman

56. Error Handling

UCab Premium implements consistent, predictable error handling across both the backend and frontend, ensuring that failures are reported clearly rather than causing silent or unhandled crashes.

56.1 Backend Error Handling

Every controller function wraps its logic in a try-catch block, as shown throughout Chapter 28, ensuring that unexpected errors are caught and returned as a structured JSON response rather than crashing the server. In addition, a centralised error-handling middleware catches any errors not already handled within a controller.

```
// middleware/errorHandler.js
function errorHandler(err, req, res, next) {
  console.error(err.stack);
  const statusCode = err.statusCode || 500;
  res.status(statusCode).json({
    success: false,
    message: err.message || 'An unexpected server error occurred.',
  });
}

module.exports = errorHandler;
```

Code 56.1: Centralised error-handling middleware, registered last in server.js

56.2 Common Error Scenarios

Scenario	HTTP Status	Handling Approach
Missing or invalid JWT token	401 Unauthorized	Caught by authMiddleware before reaching the controller.
Insufficient role permissions	403 Forbidden	Caught by roleMiddleware before reaching the controller.
Requested resource not found (e.g. invalid booking ID)	404 Not Found	Explicitly checked and returned within the relevant controller.
Invalid or incomplete request data	400 Bad Request	Caught by Mongoose schema validation or explicit controller-level checks.
Attempted assignment of an unavailable driver	400 Bad Request	Rejected by business-rule validation in the Driver Assignment Module.
Unexpected server/database failure	500 Internal Server Error	Caught by the centralised errorHandler middleware.

Table 56.1: Common error scenarios and their handling across the UCab Premium API

56.3 Frontend Error Handling

On the frontend, API errors are caught at the point of each Axios call and surfaced to the user through inline form errors or toast-style notifications, rather than failing silently. As described in Chapter 39, the global response interceptor additionally handles session-expiry errors (401 responses) uniformly across the entire application, automatically redirecting the user to the login page.

57. Screens Explanation

This chapter presents the key screens of UCab Premium across its Rider, Driver, and Admin modules. Each screen below is illustrated with an actual application screenshot alongside a professional explanation of its purpose and layout.

57.1 Rider Module Screens

Login / Registration Screen

This screen presents a clean, centred authentication form against the platform's dark navy background, with toggle links to switch between the login and registration views. Input validation feedback is shown inline beneath each field, and a prominent amber call-to-action button submits the form.

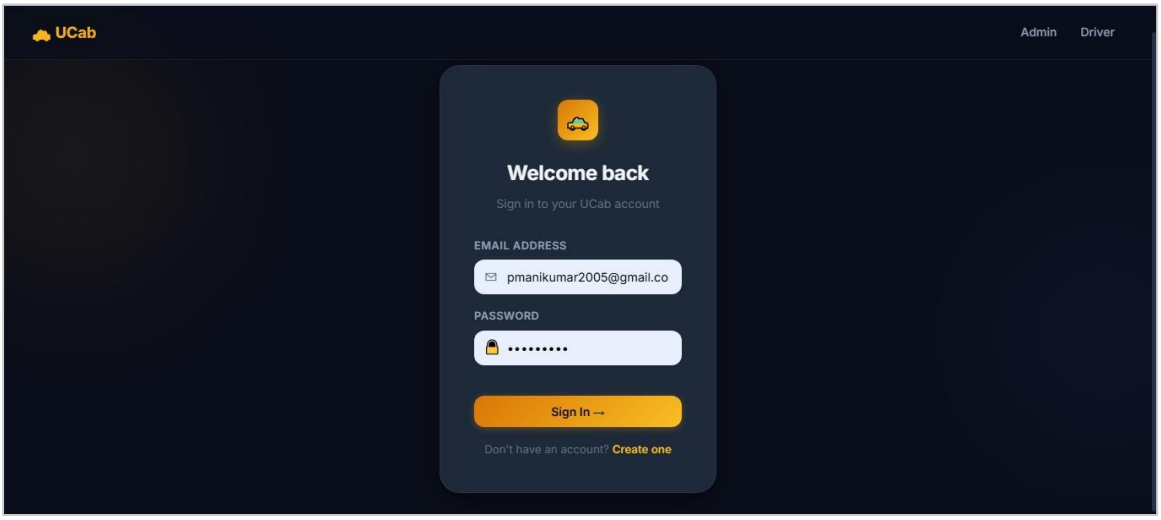

Figure 57.3: Login / Registration screen

Book a Ride Screen

The booking screen presents two primary input fields for pickup and drop location, alongside a clearly visible “Book Ride” action button. Upon successful submission, a confirmation state is shown, reflecting the newly created booking's Requested status.

Ride History Screen

This screen displays a scrollable list of BookingCard components (Chapter 36), each showing the route, a colour-coded status badge, and the fare. Completed rides additionally display a receipt download icon, linking to the PDF generated by the backend.

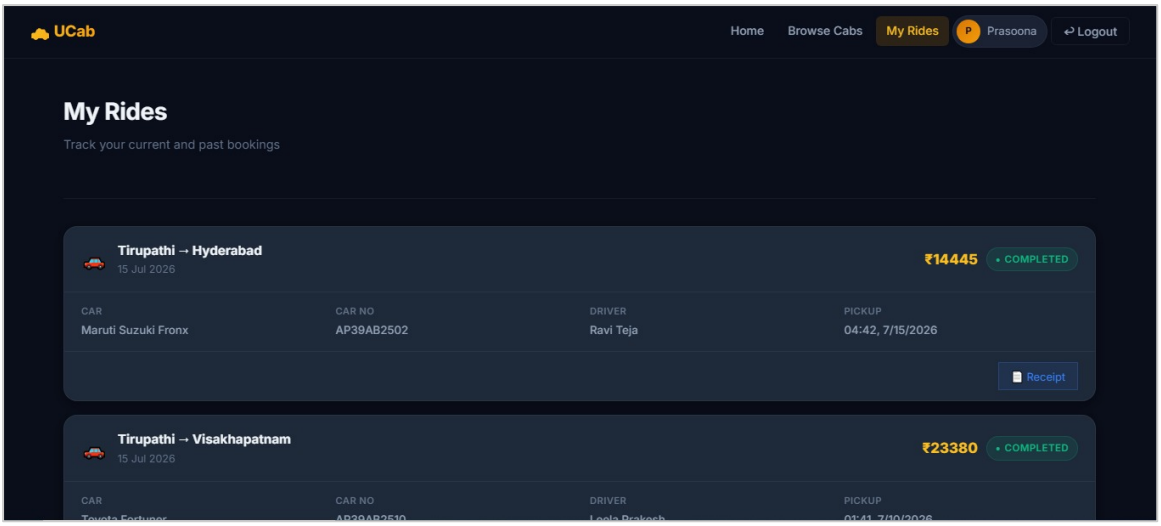

Figure 57.1: Ride History screen showing booking cards with route, status, and fare

57.2 Driver Module Screens

Driver Dashboard

The driver dashboard prominently features the availability toggle switch at the top of the screen, allowing the driver to move between Available and Offline at a glance. Below it, a summary of the driver's currently assigned ride, if any, is displayed with quick-access action buttons to start or complete the trip.

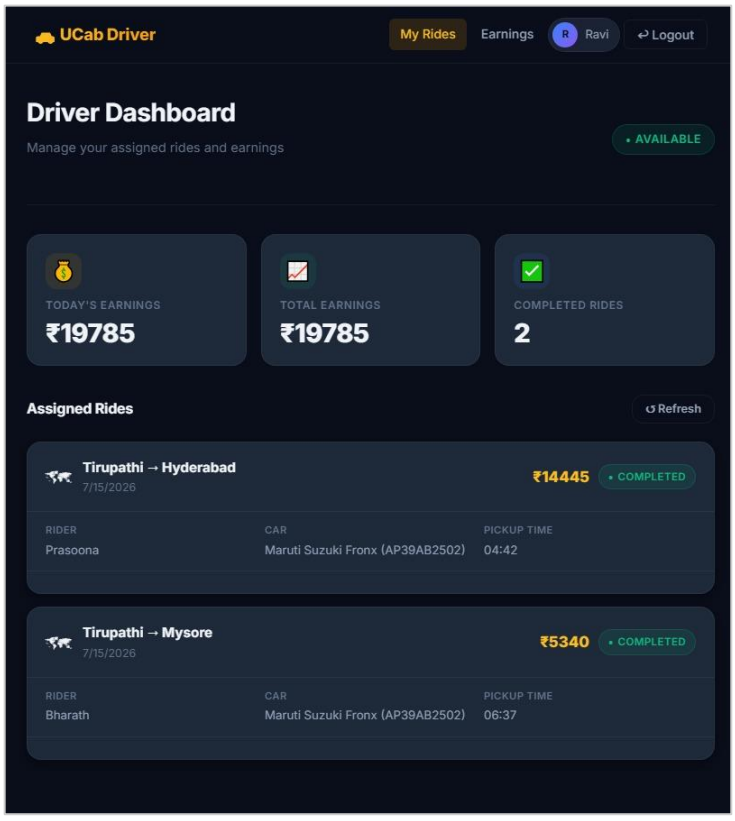

Figure 57.7: Driver Dashboard with earnings summary and assigned rides

Assigned Rides Screen

This screen lists all rides currently or previously assigned to the driver, with the active ride visually distinguished at the top of the list. Status transition buttons (Start Trip, Complete Trip) are conditionally rendered based on the ride's current lifecycle stage.

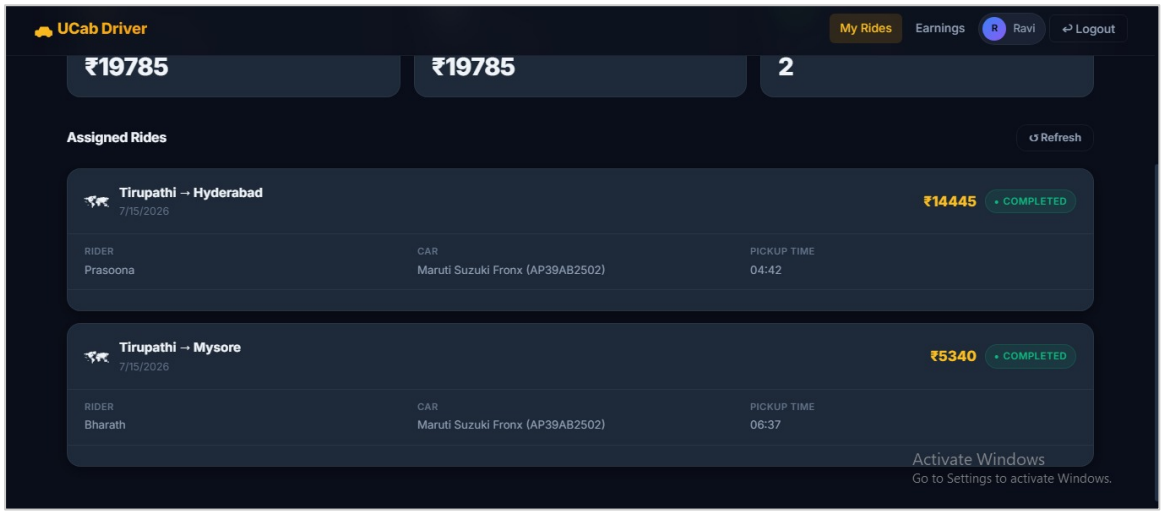

Figure 57.4: Assigned Rides screen showing a driver's completed and active rides

57.3 Admin Module Screens

Admin Dashboard

The admin dashboard is the most data-dense screen in the application, presenting the ten statistical summary cards (such as Total Bookings, Active Drivers, and Revenue) in a responsive grid at the top, followed by Recharts-based line and bar charts visualising booking trends and driver performance over time.

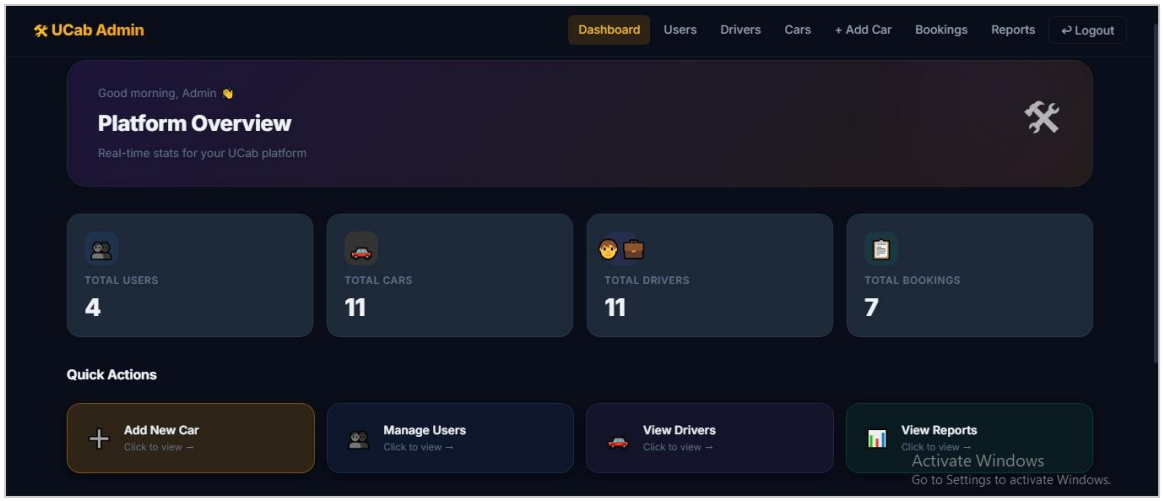

Figure 57.2: Admin Dashboard with platform statistics and quick actions

Driver Assignment Screen

This screen presents a pending booking's details alongside a filtered list of currently available drivers, each shown with their name, vehicle details, and rating. A single confirmation action assigns the selected driver, immediately updating both the booking and driver records.

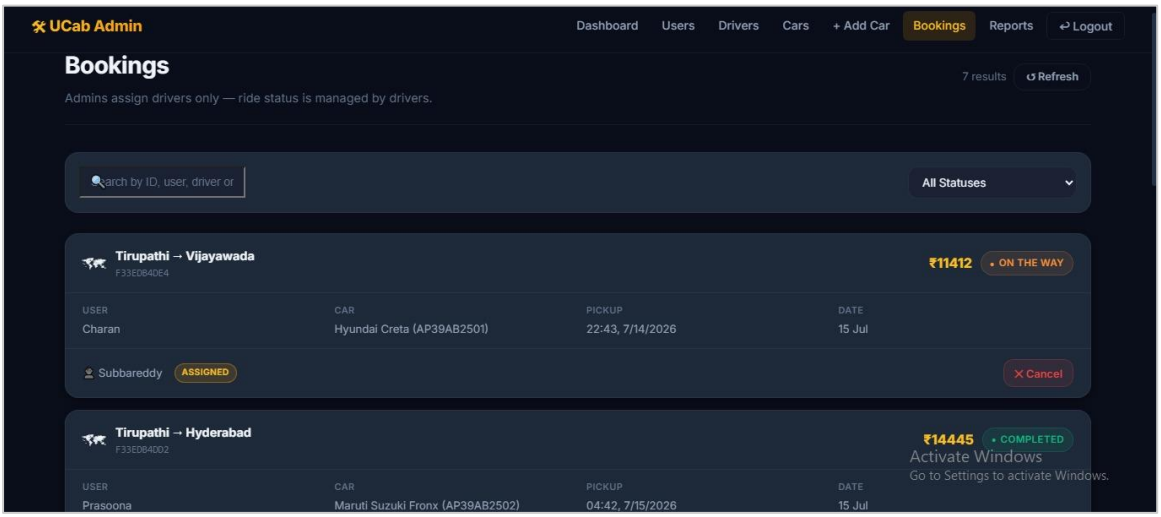

Figure 57.5: Admin Bookings screen showing driver assignment and status

Manage Drivers Screen

This screen provides a searchable, filterable table of all registered drivers, showing their availability status, license number, and associated vehicle. Administrators can use the search and filter tools described in Chapter 45 to quickly locate specific drivers.

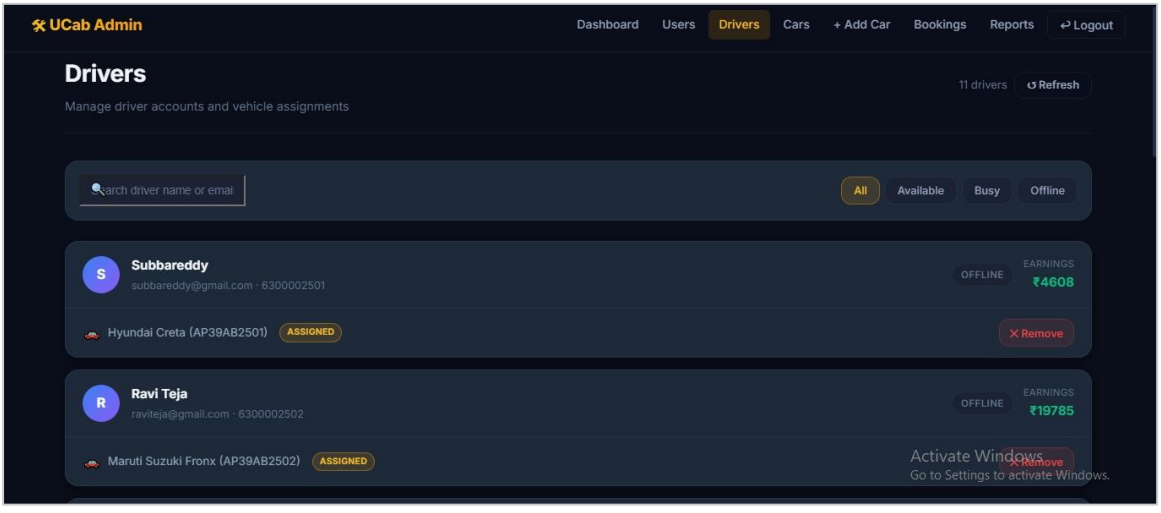

Figure 57.8: Manage Drivers screen with search and status filters

57.4 Summary of Application Screens

Module	Pages Represented Above	Total Pages in Module

Module	Pages Represented Above	Total Pages in Module
Rider	Login/Registration, Book a Ride, Ride History	8
Driver	Driver Dashboard, Assigned Rides	7
Admin	Admin Dashboard, Assign Driver, Manage Drivers	8

Table 57.1: Representative screens shown against the total page count of each module (Chapter 37)

58. Advantages

The design and implementation choices made throughout UCab Premium's development yield a number of tangible advantages over both the manual booking processes described in Chapter 3 and simplified academic prototypes that omit real-world operational constraints.

- Prevention of double-booking: The strict availability-based driver assignment logic, described in Chapter 50, structurally prevents a driver from being assigned to two overlapping bookings.
- Strong credential security: All passwords are hashed using bcryptjs, and all sessions are governed by short-lived, signed JWT tokens, following industry-standard authentication practices.
- Consistent data integrity: The five-stage booking lifecycle state machine, enforced both at the schema and controller level, ensures that a booking's status always accurately reflects a valid, real-world stage of the ride.
- Operational visibility: The administrator dashboard's ten statistical summary cards and Recharts-based visualisations give operators immediate insight into platform activity without manual computation.
- Automated proof of service: PDF receipts are generated automatically upon ride completion, improving transparency and trust without requiring any manual intervention.
- Maintainable, modular codebase: The consistent application of the MVC pattern across the backend, and a component-based structure across the 23 React pages, keeps the codebase easy to extend and debug.
- Cost-effective technology choices: The entire system was built using free-tier and open-source tools, as established in the feasibility study in Chapter 11, requiring no licensing expenditure.
- Scalable cloud-hosted data layer: MongoDB Atlas removes the burden of local database server management while supporting future growth in data volume.

59. Limitations

As with any project scoped and delivered within a fixed internship timeframe, UCab Premium carries a number of limitations, most of which reflect deliberate scope decisions rather than technical shortcomings, as outlined in the Scope of the Project (Chapter 6).

- No live GPS tracking: The system does not provide real-time map-based tracking of drivers or turn-by-turn navigation, relying instead on textual pickup and drop locations.
- No integrated payment gateway: Fare amounts are recorded on each booking, but no third-party payment gateway is integrated to process actual monetary transactions.
- Static fare structure: The system does not implement dynamic, demand-based fare calculation (surge pricing) that commercial ride-hailing platforms typically employ.
- Web-only delivery: UCab Premium is delivered as a responsive web application rather than as native iOS or Android mobile applications.
- Manual driver assignment: Driver-to-booking matching is administrator-controlled rather than fully automated through proximity-based or algorithmic matching.
- Limited automated test coverage: While unit, integration, and API testing were carried out (Chapter 55), the automated test suite does not yet achieve the exhaustive coverage expected of a production-grade commercial system.
- No push or SMS notification system: Status updates are reflected within the application interface but are not additionally pushed to users via SMS or mobile push notifications.

These limitations do not detract from the project's core objective of demonstrating full-stack development competence; rather, they define a clear and realistic boundary around the current implementation, several of which are addressed as proposed directions in the following chapter.

60. Future Enhancements

Building on the architecture established in this project, several enhancements have been identified as valuable directions for extending UCab Premium beyond its current scope. These extensions span four broad categories: location and payment intelligence (live GPS tracking, Google Maps integration, and gateway-based payments), communication and safety (multi-channel notifications and an emergency SOS feature), intelligent automation (AI-based fare prediction, personalised recommendations, and automated driver matching), and production-scale infrastructure (containerised, orchestrated cloud deployment with a mature CI/CD pipeline). Table 60.1 summarises these proposed enhancements.

Enhancement	Description
Real-Time GPS Tracking	Integrating WebSocket-based location streaming (e.g. via Socket.IO) to allow riders to track their assigned driver's live location on a map.
Payment Gateway Integration	Incorporating a third-party payment gateway such as Razorpay or Stripe to support real, in-app fare payment.
Dynamic Fare Calculation	Implementing distance-based and demand-based (surge) pricing algorithms in place of the current static fare model.
Native Mobile Applications	Developing dedicated iOS and Android applications using React Native, reusing much of the existing business logic and API layer.
Automated Driver Matching	Introducing proximity- and rating-based algorithmic matching to reduce reliance on manual administrator assignment.
Push and SMS Notifications	Integrating a notification service to alert riders and drivers of status changes outside the application interface.
Multi-Language Support	Adding internationalisation (i18n) to support riders and drivers across different regional languages.
Expanded Automated Testing and CI/CD	Building a more comprehensive automated test suite and integrating a continuous integration and deployment pipeline.
Google Maps API Integration	Replacing the current static location fields with the Google Maps Platform (Places, Directions, and Distance Matrix APIs) for address autocomplete, route visualisation, and accurate distance/time-based fare estimation.
AI-Based Fare Prediction	Applying machine learning models trained on historical trip data (distance, time of day, demand patterns) to predict fares more accurately than the current static and surge-based rules.
Ride Sharing and Pooling	Allowing multiple riders travelling along overlapping routes to share a single ride, reducing per-rider cost and improving driver utilisation.
Driver Ratings and	Introducing a post-ride rating and review mechanism so riders can rate

Enhancement	Description
Feedback	drivers (and vice versa), with aggregated scores feeding into the automated driver-matching algorithm.
Email Notification System	Sending transactional emails (booking confirmation, receipt, status updates) via a service such as Nodemailer or SendGrid, complementing the existing in-app and planned push/SMS alerts.
Emergency SOS Feature	Adding an in-ride SOS button that shares the rider's live location with emergency contacts and platform administrators in case of a safety incident.
Voice-Based Booking	Supporting ride booking through voice commands and natural-language input, improving accessibility and hands-free usability.
Machine Learning-Based Recommendations	Using ride history and usage patterns to recommend frequent destinations, optimal pickup points, and personalised ride suggestions to riders.
Containerised Cloud Deployment	Packaging the frontend and backend using Docker, orchestrating multi-container deployment with Kubernetes, and extending the CI/CD pipeline for automated, zero-downtime cloud releases.

Table 60.1: Proposed future enhancements for UCab Premium

61. Conclusion

UCab Premium was conceived and delivered as the capstone project of a Full Stack Development Internship on the SmartBridge platform, and successfully demonstrates the practical application of the MERN Stack — MongoDB, Express.js, React.js, and Node.js — to a functionally complete, multi-role transportation booking system. Over the course of the internship, from May 2026 to July 2026, the project progressed from an initial system establishing the core booking lifecycle and driver availability state machine, through to a refined, analytics-driven platform featuring automated PDF receipts, a comprehensive administrator dashboard, and a cohesive, premium user interface.

The project required the integration of a broad range of full-stack development competencies: secure, JWT-based authentication and bcryptjs password protection; a normalised, relationally-aware MongoDB data model spanning five collections; an MVC-structured Express backend exposing 35 RESTful API endpoints; and a 23-page, component-driven React frontend supporting three distinct user roles. Beyond the purely technical implementation, the project also demanded careful attention to system design, feasibility analysis, and the operational realities of a ride-booking workflow — particularly the strict enforcement of driver availability to prevent double-booking, and the disciplined, state-machine-driven progression of every booking from request to completion.

While UCab Premium does not replicate every feature of a commercial-scale ride-hailing platform — omitting, by deliberate scope decision, live GPS tracking, payment gateway integration, and dynamic pricing — it succeeds in its principal objective: providing a functionally complete, well-architected, and professionally documented demonstration of modern full-stack web development. The skills consolidated through this project, from database design and API architecture to React state management and UI design, directly reflect the practical, industry-relevant competencies the internship was designed to build.

62. References

The following official documentation, technical resources, and standards were referred to throughout the design, development, and documentation of UCab Premium.

[1] Meta Open Source, "React – The library for web and native user interfaces," React Documentation. [Online]. Available: https://react.dev. [Accessed: Jul. 14, 2026].

[2] OpenJS Foundation, "Node.js documentation," Node.js. [Online]. Available: https://nodejs.org/docs. [Accessed: Jul. 14, 2026].

[3] OpenJS Foundation, "Express – Node.js web application framework," Express.js Documentation. [Online]. Available: https://expressjs.com. [Accessed: Jul. 14, 2026].

[4] MongoDB, Inc., "MongoDB Atlas documentation," MongoDB. [Online]. Available: https://www.mongodb.com/docs/atlas. [Accessed: Jul. 14, 2026].

[5] Automattic, "Mongoose ODM v8 documentation," Mongoose. [Online]. Available: https://mongoosejs.com/docs. [Accessed: Jul. 14, 2026].

[6] Auth0, "Introduction to JSON Web Tokens," JWT.io. [Online]. Available: https://jwt.io/introduction. [Accessed: Jul. 14, 2026].

[7] npm, Inc., "bcryptjs," npm Registry. [Online]. Available: https://www.npmjs.com/package/bcryptjs. [Accessed: Jul. 14, 2026].

[8] Axios Contributors, "Axios – Promise based HTTP client for the browser and Node.js," Axios Documentation. [Online]. Available: https://axios-http.com. [Accessed: Jul. 14, 2026].

[9] Remix Software, "React Router documentation," React Router. [Online]. Available: https://reactrouter.com. [Accessed: Jul. 14, 2026].

[10] Recharts Group, "Recharts – A composable charting library built on React components," Recharts Documentation. [Online]. Available: https://recharts.org. [Accessed: Jul. 14, 2026].

[11] PDFKit Contributors, "PDFKit – A JavaScript PDF generation library for Node.js and the browser," PDFKit Documentation. [Online]. Available: https://pdfkit.org. [Accessed: Jul. 14, 2026].

[12] Mozilla Foundation, "Web APIs and JavaScript reference," MDN Web Docs. [Online]. Available: https://developer.mozilla.org. [Accessed: Jul. 14, 2026].

[13] SmartBridge Platform, "Full Stack Development (MERN Stack) internship curriculum and project brief," internal internship documentation, May–Jul. 2026.

63. Appendix

Appendix A: List of Abbreviations

Abbreviation	Full Form
MERN	MongoDB, Express.js, React.js, Node.js
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
MVC	Model-View-Controller
CRUD	Create, Read, Update, Delete
RBAC	Role-Based Access Control
UI/UX	User Interface / User Experience
SPA	Single-Page Application
ODM	Object Document Mapper
TLS	Transport Layer Security
CI/CD	Continuous Integration / Continuous Deployment

Table 63.1: Abbreviations used throughout this report

Appendix B: Sample Environment Configuration

The following illustrates the structure of the .env configuration file used by the backend server, with placeholder values shown in place of actual secrets.

```
# server/.env (sample structure - values redacted)
PORT=5000
MONGO_URI=<mongodb_atlas_connection_string>
JWT_SECRET=<jwt_signing_secret>
JWT_EXPIRES_IN=7d
NODE_ENV=production
```

Code 63.1: Sample .env structure (actual secret values are never committed to version control)

Appendix C: Project Repository

The complete source code for UCab Premium, including the client and server directories, a professional README, and an MIT License, is maintained in the GitHub repository Cab-Booking-Premium, developed and version-controlled throughout the internship period of May 2026 to July 2026. The repository is publicly accessible at: **https://github.com/manikumar26092005/Cab-Booking-Premium.**

Appendix D: Project Team

UCab Premium was developed as part of a five-member internship team on the SmartBridge platform. Table 63.2 lists the team members and their roles; the individual contributions, implementation, and documentation described throughout this report reflect the work carried out by the author as Team Lead.

Name	Email	Role
Godugunuru Penchala Raviteja	godugunuruteja@gmail.com	Member
Puli Mokshagna	mokshapuli7@gmail.com	Member
Panthagani Rupan Raj	panthaganirupanraj@gmail.com	Member
Paidipati Manikumar	pmanikumar2005@gmail.com	Team Lead
Sai Ganesh Uday Kiran Bokkasam	udaykiranroyals9099@gmail.com	Member

Table 63.2: Project team members and roles